

A Lean Blueprint for the Okutsu–Montes Counting Machinery discharging the conditional hypotheses of `goal_theorem_montes` (v2, DAG-closed)

(draft, blueprint-first; auto-assembled for review)

Draft note (v2). Cross-references render as their label slugs. The DAG is closed: every `\uses` edge resolves and all five `MontesData` fields plus `goal_theorem_montes` are backed end-to-end (§Closing the DAG). Remaining items are documented scope boundaries (one GMN import, the degree-decrease termination substitution, the deferred discriminant-tail field) and the M8 proof obligations – see the Status appendix.

Contents

1	Overview, target, and honest scope	1
1.1	The result this blueprint formalizes	1
1.2	Target and interface	1
1.3	Module roadmap M0–M9 (one paragraph)	1
1.4	DAG overview (nodes and dependencies)	2
1.5	Honest scope	2
1.6	Notation	3
2	Target and interface	3
2.1	The theorem we make unconditional	3
2.2	The count-native semantics (no measure theory)	5
2.3	The five <code>MontesData</code> fields to discharge	5
2.4	The count-native cell datum, coefficient, and recursion (reused blocks)	7
2.5	The abstract foundation and the surviving objects	7
3	Layer 0 — Finite-count foundations (M0 ResidueCount, M1 QuotientBox, M2 PadicValuation)	8
3.1	M0 — ResidueCount: the residual-count oracle	9
3.2	M1 — QuotientBox: $/\mathfrak{p}^N$ as the finite ring (p^N)	10
3.3	M2 — PadicValuation: coefficient valuations and the support Finset	12
4	Layer 1: Newton polygons and residual polynomials (the new geometry)	13
4.1	M3: the lower Newton polygon	13
4.2	M4: the residual polynomial of a side	16
5	Layer 2: the recursive OM type and its termination (M5, M6)	18
5.1	M5: the order- r OM type	18
5.2	M6: well-founded termination by degree descent	20
5.3	Discharge of the <code>MontesData</code> obligations	22

6	Layer 3 — The level-N counting bijection and the real instance (M7–M9)	22
6.1	M7 — <code>CellPartition</code> : real cells and the exhaustive partition	23
6.2	M8 — <code>LevelNCount</code> : the counting bijection (the long pole)	24
6.3	M9 — <code>RealInstance</code> : assemble and fire the goal	27
7	Closing the DAG (v2): the six glue nodes	30
7.1	GAP 1 (M4): the complete coprime residual factorization	30
7.2	GAP 4 (M3–M7): the per-side L4 lattice face	32
7.3	GAP 2 (M5/M7/M9): the <code>ClusterShape</code> → <code>OMType</code> transport	33
7.4	GAP 3 (M5): the CITED GMN dissection + OM-tree uniqueness	35
7.5	GAP 5 (M8): the finite certification level $N_0(c)$	37
7.6	GAP 6 (M8): the self-loop ratio resums to <code>countPivot</code>	38
7.7	Canonicalization of labels (v1 → v2)	39
7.8	Notation fixes (v2)	40
A	Status (v2 closure verdict)	42
S.1	DAG closure verdict	42
S.2	Residual scope boundaries (not DAG gaps)	42
S.3	Lean-ing order (from v1 reconciliation R.6)	42

1 Overview, target, and honest scope

1.1 The result this blueprint formalizes

We give a leanblueprint-style plan for a Lean 4 (v4.31.0 / mathlib v4.31.0) formalization of the Okutsu–Montes / Guardia–Montes–Nart (GMN) machinery, whose single deliverable is to make `LeanUrat.Goal.goal_theorem_montes_unconditional` — i.e. *to replace its carried hypotheses (M,D,F,hbridge) by constants in the count field) outstanding. The framing is count-native, no measure theory*: p -adic density is the counting–fraction limit over $\mathcal{O}_K/\mathfrak{p}^N \cong (p^N)$ (a finite ring via `PadicInt.toZModPow`); the box count is the finite cardinality $(p^N)^n$; the density of a type σ is $\lim_N \#\{\text{cosets decided to } \sigma\} / (p^N)^n$. The combinatorial core — Newton polygons, residual polynomials, OM types and their descent, termination, and the level- N counting bijection — is built bottom-up under `LeanUrat/OM/` in ten modules M0–M9. The measure route (`PadicMeasure.clusterMeasure`, `AX_ellRecursion`, `omCells`, `descend_size`) is bypassed entirely.

1.2 Target and interface

The theorem fired at the top of the build is reproduced verbatim from `Goal.lean:149--189`:

```
theorem goal_theorem_montes {q n : ℕ} (M : CountingModel q n) (D : MontesAxiom.MontesData
q n M) (σ : FactorizationType) (F : DensityFoundation) (hσ : σ.degree = n) (hbridge
: ∀ q' : ℕ, 1 < q' → F.density n σ q' = ∑ T ∈ D.shapesOf σ, D.coeff T q') :
∃ (num den : Polynomial ℚ), den ≠ 0 ∧ ... ∧ IsPalindromic num den
```

The five `MontesData` fields the OM build must *construct/prove* (their types quoted verbatim from `MontesAxiom.lean:389--504`) are the discharge targets of this blueprint: `partition`, `cells_descend`, `finiteTermination`.

1.3 Module roadmap M0–M9 (one paragraph)

M0 ResidueCount re-exports the proved finite-field counts as the residual-count oracle ($\#\{\text{non-squarefree monic } Q^{m-1}, \text{ the } T_{BB3} \text{ value, } = \text{MontesAxiom.residualBoxCount}\}$). **M1 QuotientBox** models $O_K/\mathfrak{p}^N$ as the finite ring (p^N) and proves $\#\text{monicBox}(N, n) = (p^N)^n$ (the density denominator). **M2 PadicValuation** gives the coefficient valuations v_i and the support Finset. **M3 NewtonPolygon** builds the lower Newton polygon as an explicit integer hull-scan (not via mathlib convexHull), extracts vertices/sides/slopes, and bridges to L4.LatticePolygon. **M4 ResidualPolynomial** defines R_S over the finite (hence perfect) residue field, proves $\deg R_S = \ell(S)/e$ and the separable $\Leftrightarrow$ squarefree descent trigger. **M5 OMType** carries the recursive order- r type (tower, key polynomial as imported data, N_r, R_r) with the leaf/descend dichotomy. **M6 Termination** proves the strict degree-decrease and $\text{treeSize} \leq n$, discharging $\text{cells}_{\text{descend and finiteTermination}}$. **M7 CellPartitionemitscellsT** : $\text{ListCountCells} \rightarrow \text{the constant} - \text{fiberHensel} - \text{liftbijection gives stratumCount T N} = \sum_c (\text{box}) \cdot (\text{residual}) \cdot \prod_{\text{children}}$ and the density limit, discharging boxHaarEquidist and $\text{nodeMeasure}_{\text{boxSum}}$. **M9 RealInstanceassemblethereal**

1.4 DAG overview (nodes and dependencies)

The build is a DAG over the module layers; edges run upward (a node nodes below it):

- **Layer 0:** $\text{lem:nonsqfree-count}$ (M0, $\leftarrow$ L3/L3Squarefree), lem:card-monicBox (M1, $\leftarrow$ $\text{lem:card-degreeLT-fallback} \leftarrow \text{degreeLTEquiv} + \text{Nat.cardFun}$), $\text{def:val} - \text{support}$ (M2). **Layer 1:** $\text{def:np} - \text{vertices} \rightarrow \text{def:to-lattice}$ (M3, $\leftarrow$ def:val-support); $\text{def:residual-poly} \rightarrow \text{thm:residual-sep-sqfree} \rightarrow \text{lem:descent-repeated-factor}$ (M4); plus the NEW $\text{lem:residual-factorization}$ and $\text{def:side-to-l4face}$ (see scope).
- **Layer 2:** $\text{def:om-type}, \text{def:om-cell}$ (M5, $\leftarrow$ Layer 1 + cited ext:OMShape-shadow); $\text{lem:child-size-lt} \rightarrow \text{thm:treesize-le-n}$ (M6) discharging $\text{thm:cells-descend-discharge}, \text{thm:finite-termination-discharge}$.
- **Layer 3:** $\text{def:cells-of-type} \rightarrow \text{lem:m7-partition}$ (M7, $\leftarrow$ $\text{lem:residual-factorization}, \text{lem:om-tree-uniqueness}$); $\text{lem:l4-coordinate-residue-indep} + \text{lem:constant-fiber} \rightarrow \text{thm:levelN-bijection} \rightarrow \text{thm:stratum-limit}$ (M8, the long pole, discharging $\text{boxHaarEquidist} + \text{nodeMeasure}_{\text{boxSum}}$); $\text{def:real} - \text{countingmodel} + \text{def:real} - \text{montesdata} \rightarrow \text{def:real} - \text{foundation} \rightarrow \text{thm:m9-goal}$ (M9, the root).

The root thm:m9-goal additionally the CITED interface facts lem:om-engine (rationality), $\text{lem:node-mult-derived}$, $\text{lem:undecided-vanishes}$, and the surviving axiom $\text{def:surviving-axiom}$.

1.5 Honest scope

Substitution (degree-decrease, not GMN Thm 4.18). Termination (M6) is proved by the *strict degree-decrease* of the cluster-size descent measure: each descent child carries a multiplicity- ≥ 2 residual factor of cluster size $\mu \leq d_r \leq \text{cs}(T)$, with strict slack off the self-loop, so $\text{treeSize} \leq n$ by strong induction. This is a deliberate **SUBSTITUTION** for GMN’s sharp Theorem of the Index (Thm 4.18 / Cor 4.19), whose formalization (higher-order valuations v_r , index identities, defectlessness) is multi-month and out of scope. We lose only the *sharpness* of the depth bound, never its existence; the degree-budget bound is purely combinatorial and hence automatically p -independent.

Deferred (one elementary field). The discriminant-tail decay $\text{discriminantTailMeasure } N \leq C \cdot q^{-N}$ ($\text{MontesAxiom.lean:488--504}$) — a nonzero polynomial’s root-set over (p^N) has density $\leq C \cdot q^{-N}$ — is the one elementary number-theory fact absent from mathlib v4.31.0. It is carried HONESTLY as the named MontesData field $\text{discriminantTail}_{\text{envelope}}$, *never as a sorry or arelabelled analytic axiom; the 0 limit is then proved* from the envelope.

Surviving axiom. The sole non-Montes external input that survives is `tameFunctionalEquation(Interface.lean.tamepalindromic-weightmachinery.‘‘Unconditional’’meansprecisely : theaxiomfootprintofthefiredgoalis{pro`
`tameFunctionalEquation}`plusthedeferredfieldvalue.

Build-risks (carried, mitigated). (i) *Field vs. finite ring* (M1): L3’s `#degreeLT =  $Q^m$` uses `Module.natCardeqpowfinrank`, which requires `[DivisionRingK](itroustestthroughIsNoetherian.finsetBasis)`; und for $N \geq 2$. Mitigated by the fallback `lem:card-degreeLT-fallback` via `Polynomial.degreeLTEquiv` (valid over any `[Semiring R]`, confirmed) and `Nat.cardfun---`—thesameroutemathlib’sownnatCard_{e_{q_pow_{fin}rank}}—theconstant—fiberclaimrequirestheNewton—valuationandresidual—unitdigitstobegenuinelydisjointcoordin and a finite certification level $N_0(c)$ above which the per-step ratio is exactly constant; both must be made explicit nodes. (iii) *Geometric self-loop resummation* to `countPivot` risks an off-by-one in `selfLoopExponent`. (iv) *Missing glue*: the complete residual coprime factorization, the `ClusterShape $\leftrightarrow$ OMType` decode, the OM-tree uniqueness import, and the per-side L4 face are not yet nodes (see assembly order).

1.6 Notation

Symbol	Meaning
$K, O_K, \mathfrak{p}$	finite ext. of p ; integers; maximal ideal. Core: $K =_p, O_K =_{[p]}$
p, q	rational prime; residue cardinality. Core (unramified): $q = p$, so $(p^N)^n = q^{nN}$
n	degree; $f = X^n + \sum_{i < n} a_i X^i$ monic over O_K
N	truncation level; box = <code>monicBox(N, n)</code> over (p^N) , $\# = (p^N)^n$
v, v_i	<code>PadicInt.valuation</code> ($v(p) = 1$); $v_i = v(a_i)$
<code>npHeight, S</code>	lower Newton height; a side S of length $\ell(S)$
$\lambda_S = -h_S/e_S$	side slope in lowest terms; e_S the Newton-side ramification denominator
$R_S, k_S = F_r$	residual polynomial over the finite residue field; $\deg R_S = \ell(S)/e_S$
T, r	OM type; order. Tower $F_0 \subseteq \dots \subseteq F_r$, key poly ϕ_r , residual R_r
δ	residue-extension degree $[F_r : F_0]$ (<code>CountCell</code> field); residual count over F_{q^δ}
$d_S = \deg \psi$	residual-stratum degree (<code>CountCell</code> field d_S); descent count needs $d_S \geq 2$
<code>cs(T) = treeSize</code>	cluster size / well-founded descent measure (= factor mult. at descent)
$w = \text{selfLoopExponent}(s)$	$s(s+1)/2 - 1$; distinct from e_S (do not conflate)
<code>countCellCoeff c</code>	$((q)^\delta)^{d_S-1} \cdot L4.bb1Value\ c.\text{polygon}$
<code>countPivot s</code>	$1 - (q^{w(s)})^{-1}$ if $s \geq 2$, else 1 (self-loop resummation)
$g_\sigma(q)$	$\sum_{T \in \text{shapesOf } \sigma} D.\text{coeff } T\ q$ (note coeff lives in Decomposition , =D.C)

2 Target and interface

This section fixes the *source of truth* for the whole formalization: the single theorem we make unconditional, the exact hypothesis bundle it consumes, the five **MontesData** fields the OM machinery (modules M0–M9) must discharge, the count-native semantics in which everything is phrased (no measure theory), and the two objects that deliberately survive. Every node below is read *verbatim* from the existing `lean.urat` sources (file:line cited); later sections reference these labels and the reconciler checks the DAG closes against them. Nothing here is NOVEL: this is the *interface boundary*, all CITED to existing Lean declarations.

2.1 The theorem we make unconditional

Definition 2.1 (The goal theorem, verbatim). **Status: CITED (reused verbatim).** Source: `LeanUrat/Goal.lean:149--189`, `[lean: LeanUrat.Goal.goal‘‘theorem‘‘montes]`. We reuse this declaration

unchanged; the deliverable of the whole OM build is to *construct its hypotheses* (`def:counting-semantic, def:montes`) from genuine p -adic combinatorics rather than carrying them abstractly. Its exact shape is:

```

theorem goal_theorem_montes
  {q n : ℕ} (M : CountingModel q n) (D : MontesAxiom.MontesData q n M)
  (σ : FactorizationType) (F : DensityFoundation) (hσ : σ.degree = n)
  (hbridge : ∀ q' : ℕ, 1 < q' →
    F.density n σ q' = ∑ T ∈ D.shapesOf σ, D.coeff T q') :
  ∃ (num den : Polynomial ℚ), den ≠ 0 ∧
  (∀ q' : ℕ, 1 < q' → den.eval (q' : ℚ) ≠ 0 ∧
    (∑ T ∈ D.shapesOf σ, D.coeff T q') = num.eval (q' : ℚ) / den.eval (q' : ℚ))
  ∧
  (1 < q → M.countingDensity σ = num.eval (q : ℚ) / den.eval (q : ℚ)) ∧
  IsPalindromic num den

```

The hypothesis bundle is **five inputs**: $M : \text{CountingModel } q \ n$ (`def:counting-semantic`), $D : \text{MontesData } q \ n \ M$ (`def:montesdata-fields`), the type σ , an abstract $F : \text{DensityFoundation}$ (`def:density-foundation`), the degree witness $h\sigma$, and the *bridge hypothesis* $hbridge$ that identifies F 's abstract density with the count-native OM tree-sum $g_\sigma(q') := \sum_{T \in \text{shapesOf } \sigma} D.\text{coeff } T \ q'$ at every prime power $q' > 1$. The conclusion is a single $\text{num}/\text{den} \in \mathbb{Q}(t)$ that (i) uniformly represents g_σ at every $q' > 1$, (ii) takes the genuine counting density $M.\text{countingDensity } \sigma$ as its value at this model's q , and (iii) is palindromic in the faithful semantic sense (`def:palindromy`).

Theorem 2.2 (What “unconditional” means after the OM build). **Status: SUBSTITUTION (target restatement)**. After modules M0–M9 the only objects fed to `def:goal-montes` are *constructed*, not assumed: M9 (`def:real-instance-target`) builds a concrete M, D, F and a $hbridge$ proof by `rfl`, so the firing of `goal_theorem_montes` yields a statement about the *genuine* count-native density whose remaining axiomatic dependence is exactly `{propext, Classical.choice, Quot.sound, tame}` (`def:surviving-axiom`). In particular the *conditional* `MontesData` hypotheses currently carried in `lean_urat` are discharged: the five fields of `def:montesdata-fields` become *proved* from Newton-polygon / residual-polynomial / OM-descent combinatorics over $\mathbb{Z}\text{Mod}(p^N)$.

Proof sketch. This is not a new proof but a statement of the build target; the mechanism is the existing body of `goal_theorem_montes` (`Goal.lean:165--189`), which we reuse verbatim:

1. *Rationality.* `D.countingDensity_isRational σ` (`Decomposition.lean:185`, [`lean: LeanUrat.MontesAxiom.MontesData.countingDensity` “`isRational`”]) produces `num, den` with `den ≠ 0` and the uniform representation of g_σ ; this is itself derived through the generic engine `OMInduction.clusterVol_isRational` (`lem:om-engine`) over the count-native blocks.
2. *Density value.* `D.countingDensity_eq_sum_coeff σ` (`Decomposition.lean:104`) rewrites $M.\text{countingDensity } \sigma = g_\sigma(q)$, so the single number equals the rational function's value at q .
3. *Palindromy transfer.* `tame_functionalEquation F n σ hσ` (`def:surviving-axiom`) supplies a palindromic tame representative `tnum/tden` of $F.\text{density } n \ \sigma$; via `hbridge`, `num/den` also represents $F.\text{density } n \ \sigma$ at every $q' > 1$, so `L7.tame_to_all_primes` gives `num·tden = tnum·den` in $\mathbb{Q}(t)$ (identity theorem on the infinite tame prime set), and `L7.isPalindromic_of_agree` transfers palindromy.

The OM build (M0–M9) does not touch this body; it removes the *conditionality* by producing the M , D , F , `hbridge` arguments as theorems over real combinatorics. Hard point: the `hbridge` identification is the only interpretive content M9 must supply by `rfl` (M9 sets `F.density n σ := g_σ`, exactly as the witness template `Witness.trivF` does, `Witness.lean:279--296`). $\square$

2.2 The count-native semantics (no measure theory)

Definition 2.3 (Counting model: density as the decided-limit, box as $(p^N)^n$). **Status: CITED.**

Source: `LeanUrat/CountingModel.lean:95--151`, [`lean: LeanUrat.CountingModel.CountingModel`]. A `CountingModel q n` (q the residue cardinality, a prime power $q \geq 2$; n the degree) carries the level- N *certified-by-digits* counts and the elementary constraints that make the decided limit well-defined. The load-bearing fields are:

- `decidedCount : FactorizationType → ℕ → ℚ` — $\#\{\text{cosets mod } p^N \text{ decided to be type } \sigma\}$ (every lift has type σ);
- `undecidedCount : ℕ → ℚ` — $\#\{\text{cosets not yet decided for any type}\}$;
- `typeMenu : Finset FactorizationType` with `typeMenu.degree` (only degree- n types) and `decidedCount_off_menu`;
- `box_partition : ∀ N, (∑σ ∈ typeMenu decidedCount σ N) + undecidedCount N = (q : ℚ)n·N` — **this is the only place the box count $(p^N)^n = q^{nN}$ appears, and only as the partition TOTAL, never as a per-type density;**
- `decided_mono`, `densityVal`, `decided_tendsto` (the normalized decidedMeasure $\sigma N := \text{decidedCount } \sigma N / q^{nN}$ is monotone and tends to `densityVal σ`), and `upper_tail` (the per-level tail bound).

The semantic heart is `countingDensity σ := densityVal σ` (`CountingModel.lean:237`, [`lean: LeanUrat.CountingModel.countingDensity`]), *the level- N decided limit*, with `density_isLimit` proving `Tendsto (decidedMeasure`

Lemma 2.4 (Undecided vanishes is already proved from M1). **Status: CITED (proved).**

Source: `MontesAxiom.lean:621--631`, [`lean: LeanUrat.MontesAxiom.MontesData.undecidedVanishes`]. For any $D : \text{MontesData } q n M$, `UndecidedVanishes M` holds, i.e. `Tendsto M.undecided atTop (ℕ 0)` (`CountingModel.lean:349`, [`lean: LeanUrat.CountingModel.UndecidedVanishes`]). This is *not* a gap the OM build must reopen: it is discharged by the elementary squeeze $0 \leq \text{undecided } N \leq \text{separableUndecidedMeasure } N$ `discriminantTailMeasure N` (`undecided_le_split`), with both tails $\rightarrow 0$ (`sepTail_tendsto` from `finiteTermination`; `discriminantTail_tendsto` from the q^{-N} envelope). The OM build’s job is only to supply the `MontesData fields` that feed this; the limit itself is already a theorem.

Proof sketch. Verbatim the existing body: `Filter.Tendsto.add` of `sepTail_tendsto` and `discriminantTail_tendsto` gives the sum $\rightarrow 0$; then `tendsto_of_tendsto_of_tendsto_of_le_of_le'` squeezes `M.undecided` between the constant 0 (`M.undecided_nonneg`) and the sum (`undecided_le_split`). The discriminant tail uses `tendsto_pow_atTop_nhds_zero_of_lt_one` on $(q^{-1})^N$ since $q \geq 2$. No new content; cited so later modules know not to reprove it. $\square$

2.3 The five MontesData fields to discharge

Definition 2.5 (The `MontesData` structure and its discharge targets). **Status: CITED (target of the OM build).** Source: `MontesAxiom.lean:389--504`, [`lean: LeanUrat.MontesAxiom.MontesData`].

A `MontesData q n M` carries the count-native OM data `shapesOf : FactorizationType → Finset ClusterShape`, `treeSize : ClusterShape → ℕ`, `cells : ClusterShape → List CountCell` (`def:countcell`), `stratumCount : ClusterShape → ℕ → ℚ`, and a per-box measure `boxMeasure : CountCell → ℕ → ℚ`, together with the supporting tail fields. The **five fields the OM build must construct/prove** (the conditional hypotheses to discharge) are, with their EXACT types:

1. `partition` (discharged by M7/M8):

$\forall (\sigma : \text{FactorizationType}) (N : \mathbb{N}), M.\text{decidedCount } \sigma \ N = \sum T \in \text{shapesOf } \sigma, \text{stratumCount } T \ N.$

Needs the level- N *counting bijection*: the decided count of type σ is the finite sum over shapes T with $\text{type}(T) = \sigma$ of the stratum counts. Proved in M8 (`def:m8-partition-target`) from the OM strata partitioning the decided monic polynomials over $\mathbb{ZMod}(p^N)$.

2. `cells_descend` (discharged by M6):

$\forall (T : \text{ClusterShape}), \forall c \in \text{cells } T, \forall ch \in c.\text{children}, \text{treeSize } ch < \text{treeSize } T.$

Needs the well-founded strict-descent measure: every descent child has strictly smaller `treeSize`. Proved in M6 (`def:m6-termination-target`) by the total-degree decrease (child `clusterSize` = factor multiplicity; factor degrees sum $\leq$ parent residual degree $\leq$ parent degree).

3. `finiteTermination` (discharged by M6):

$\forall (\sigma : \text{FactorizationType}), \sigma.\text{degree} = n \rightarrow \forall T \in \text{shapesOf } \sigma, \text{treeSize } T \leq n.$

Needs termination with $\leq n$ leaves. Proved in M6 by strong induction on degree (SUBSTITUTION: degree-decrease well-foundedness, *not* the full GMN Thm 4.18 index bound; degree decrease suffices and is p -independent automatically).

4. `boxHaarEquidist` (discharged by M8):

$\forall (c : \text{CountCell}) (q' : \mathbb{N}), 1 < q' \rightarrow \text{boxMeasure } c \ q' = \text{countCellCoeff } c \ q'.$

Needs the per-box normalized count to equal `countCellCoeff c q' = (q')ds-1 · L4.bb1Value c.polygon q'` (`def:countcell-coeff`). Proved in M8 (`def:m8-boxhaar-target`): the residual factor is the proved finite-field count `residualBoxCount` (M0, `lem:m0-residual-count`), the box factor is the proved lattice count `L4.cellVolume_eq` re-proven as coordinate-residue independence over $\mathbb{ZMod}(p^N)$.

5. `nodeMeasure_boxSum` (discharged by M8, *the long pole*):

$\forall (T : \text{ClusterShape}), \text{Filter.Tendsto } (\text{fun } N \Rightarrow \text{stratumCount } T \ N / (q : \mathbb{Q})^{(n \cdot N)}) \text{Filter.atTop } (\text{nhds } (((\text{cells } T).\text{map } (\text{fun } c \Rightarrow \text{boxMeasure } c \ q * (c.\text{children}.\text{map } (\text{fun } ch \Rightarrow \text{clusterCount } \text{cells } \text{treeSize } \text{cells_descend } ch \ q)).\text{prod})).\text{sum}) / \text{countPivot } (\text{treeSize } T) \ q)).$

Needs the box-additive node limit: $\lim_N \text{stratumCount } T \ N / q^{nN}$ exists and is the per-box sum over the cell partition, self-loop resummed by the geometric pivot. Proved in M8 (`def:m8-boxsum-target`) by the Hensel-lift coset argument: fibers of $\mathbb{ZMod}(p^{N+1}) \rightarrow \mathbb{ZMod}(p^N)$ over a fixed cell have constant size, giving the limit $\rightarrow \text{clusterCount}$.

The remaining `MontesData` fields (`stratumCount_nonneg`, `separableUndecidedMeasure` and its tail data, `discriminantTailConst` / `discriminantTail_envelope`) are elementary count nonnegativities and the one DEFERRED elementary bound (`def:deferred-discriminant`); they are supplied alongside the five.

Lemma 2.6 (Per-node multiplicativity is a theorem given the five fields). **Status: CITED (proved from the fields)**. Source: `MontesAxiom.lean:545--564`, [lean: `LeanUrat.MontesAxiom.MontesData.nodeMultipli`]

Once `boxHaarEquidist` and `nodeMeasure_boxSum` are supplied, the per-node count factorization $\lim_N \text{stratumCount } T \, N / q^{nN} = \text{clusterCount cells treeSize cells_descend } T \, q$ is *derived* (not assumed): `nodeMeasure_boxSum` gives the limit as a box-sum over `boxMeasure`; `boxHaarEquidist` rewrites each `boxMeasure c q = countCellCoeff c q`; `clusterCount_boxSum` (`MontesAxiom.lean:363--375`, `PROVED List.sum` arithmetic) recombines into `clusterCount`. The OM build need not reprove this; supplying the five fields suffices.

Proof sketch. Verbatim the existing body: set `hbox := D.nodeMeasure_boxSum T`; rewrite the box-sum via `clusterCount_boxSum` with the per-box function $c \mapsto \text{boxMeasure } c \, q \cdot \prod_{\text{children}} \text{clusterCount}$, discharging its hypothesis by `D.boxHaarEquidist c q hq` (where `hq : 1 < q` from `M.hq`); then `rwa` folds `hbox` into the `clusterCount` target. The only inputs are the two box-wise fields plus the `PROVED` arithmetic `clusterCount_boxSum`. $\square$

2.4 The count-native cell datum, coefficient, and recursion (reused blocks)

Definition 2.7 (The pure-counting OM cell). **Status:** **CITED**. Source: `MontesAxiom.lean:100--108`, [`lean: LeanUrat.MontesAxiom.CountCell`]. A `CountCell` records one residual-stratum on one order- r Newton-polygon face: `dS : ℕ` (residual-stratum degree d_S), `δ : ℕ` (residue-extension degree, residual count over F_{q^δ}), `polygon : L4.LatticePolygon` (the lattice Newton-polygon face feeding the box count), `children : List ClusterShape` (the multi-child descent list, each strictly smaller). `M7 (def:m7-cells-target)` constructs `cells T` as a list of these from the real OM type T .

Definition 2.8 (Count-native cell coefficient and self-loop pivot). **Status:** **CITED**. Source: `MontesAxiom.lean:119--127`. The per-cell coefficient `countCellCoeff c := q ↦ ((q : ℚ)c.δ)c.dS-1`. `L4.bb1Value c.polygon q` ([`lean: LeanUrat.MontesAxiom.countCellCoeff`]) is the residual count $T_{BB3} = (q^\delta)^{d_S-1}$ times the Newton-box count $T_{BB1} = \text{bb1Value}$. The self-loop pivot `countPivot s := q ↦ if 2 ≤ s then 1 - ((q : ℚ)L5fix.selfLoopExponent s)-1 else 1` ([`lean: LeanUrat.MontesAxiom.countPivot`]). Both are `PROVED` rational (`countCellCoeff_isRational`, `countPivot_isRational`) and pole-free on $q > 1$ (`countPivot_ne` via `L5fix.selfLoop_geometric`). These are inputs the OM build reuses; it does not redefine them.

Lemma 2.9 (The generic OM-order rationality engine). **Status:** **CITED (proved)**. Source: `OMInduction.lean:34--111`, [`lean: LeanUrat.OMInduction.clusterVol "isRational"`]. A function defined by a well-founded recursion over a `size : Shape → ℕ` with rational coefficients and a pole-free self-loop pivot is a uniform rational function of q : given `hrec` (the divided sum-of-products unfold), `hdescend` (strict size decrease of children), `hcoeff` (`IsRationalFn` of each coefficient), `hpivot_rat`, `hpivot_ne`, it concludes $\forall T, \text{IsRationalFn}(\text{clusterVol } T)$. The rationality half of `def:goal-montes` flows through this engine instantiated over `ClusterShape` with the count-native data (`Decomposition.lean:135--175`); the OM build supplies the data (M5–M7) but reuses the engine.

Proof sketch. Cited verbatim: strong induction on `size T` via `Nat.strong_induction_on`; each summand is rational by `hcoeff` times `RatFn.isRationalFn_listProd` of the child volumes (rational by IH, applicable since children are strictly smaller by `hdescend`); the whole sum by `RatFn.isRationalFn_listSum`; the quotient by `RatFn.isRationalFn_div` (pole-free by `hpivot_ne`); and `congr_of_eq_gt_one` transports along `hrec` since `IsRationalFn` only constrains values at $q > 1$. $\square$

2.5 The abstract foundation and the surviving objects

Definition 2.10 (The abstract density foundation F). **Status:** **CITED**. Source: `Interface.lean:185--208`, [`lean: LeanUrat.DensityFoundation`]. $F : \text{DensityFoundation}$ carries `density : (n : ℕ) → FactorizationType →`

$(q : \mathbb{N}) \rightarrow \mathbb{Q}$, `clusterVolume`, `shapes`, `multiplicity`, the structural decomposition, and `density_nonneg`. In `def:goal-montes` F enters only through `tame_functionalEquation` (which is stated about $F.\text{density}$) and through `hbridge` (which couples $F.\text{density } n \sigma$ to the count-native g_σ). M9 constructs the real F with $F.\text{density } n \sigma := g_\sigma$ so that `hbridge` holds by `rfl`, following the template `Witness.trivF` (`Witness.lean:279--296`).

Definition 2.11 (Faithful semantic palindromy). **Status: CITED.** Source: `Interface.lean:315--317`, [`lean: LeanUrat.IsPalindromic`]. `IsPalindromic num den` is the degree-robust functional equation $R(1/x) = R(x): \forall x : \mathbb{Q}, x \neq 0 \rightarrow \text{den.eval } x \neq 0 \rightarrow \text{den.eval } x^{-1} \neq 0 \rightarrow \text{num.eval } x^{-1} / \text{den.eval } x^{-1} = \text{num.eval } x / \text{den.eval } x$. It is the conclusion’s palindromy clause and is supplied by the tame axiom plus the L7 transfer; the OM build does not touch it.

Definition 2.12 (The one surviving axiom: tame functional equation). **Status: DEFERRED (cited axiom, intentionally retained).** Source: `Interface.lean:326--332`, [`lean: LeanUrat.tame “functionalEquation` The named axiom asserts that at tame primes $p > n$ the projective density $F.\text{density } n \sigma$ is represented by a palindromic $\text{num}/\text{den} \in \mathbb{Q}(t)$:

```
axiom tame_functionalEquation (F : DensityFoundation) :
  ∀ (n : ℕ) (σ : FactorizationType), σ.degree = n →
  ∃ (num den : Polynomial ℚ), den ≠ 0 ∧
  (∀ q : ℕ, q.Prime → n < q → F.density n σ q = num.eval (q:ℚ) / den.eval
  (q:ℚ)) ∧
  IsPalindromic num den
```

This is the paper’s §5 H-tame palindromic-weight machinery and is the **sole non-Montes external input that survives** the OM build; making the goal unconditional is “modulo `tame_functionalEquation`.” Citation: `notes/HUMAN_PROOF.md §12`, paper §5, Del Corso–Dvornicich (2000), Yin.

Definition 2.13 (The one deferred elementary field: discriminant-tail decay). **Status: DEFERRED (elementary, named field).** Source: `MontesAxiom.lean:488--504` (`discriminantTailConst`, `discriminantTailConst_nonneg`, `discriminantTail_envelope : ∀ N, discriminantTailMeasure N ≤ discriminantTailConst · ((q : ℚ)N)-1`). The non-separable (discriminant-zero mod p^N) locus has counting density $\leq C \cdot q^{-N}$: a nonzero polynomial’s root-set over $\mathbb{Z}\text{Mod}(p^N)$ has density $\leq C \cdot q^{-N}$. This explicit root-count bound over $\mathbb{Z}\text{Mod}(p^N)$ is the one elementary number-theory fact absent from `mathlib v4.31.0`; the OM build (M9) carries it as this single named field, NOT faked in a `sorry` and NOT relabeled as an analytic axiom. It is Montes / discriminant content, so it does not block the rationality footprint; the $\rightarrow 0$ limit is then PROVED from the envelope (`lem:undecided-vanishes`).

Definition 2.14 (M9 real-instance assembly target). **Status: SUBSTITUTION (the M9 deliverable, shape from Witness).** Source template: `Witness.lean:156--319` (`trivM/trivD/trivF/montes_full`). M9 assembles a REAL `CountingModel q n` ($q = p$; `decidedCount` from M8; `box total` $(p^N)^n$ via `M1/def:counting-semantics`; `decided_tendsto` from the M8 limit; `upper.tail` from Hensel monotonicity), a REAL `MontesData` (all five fields of `def:montesdata-fields` proved by M6/M7/M8), a REAL `DensityFoundation F` with `density := gσ`, and proves `hbridge` by `rfl`; then fires `goal.theorem_montes M D σ F hσ hbridge` to obtain the unconditional statement modulo `tame_functionalEquation` and the deferred field (`def:deferred-discriminant`). The `PadicMeasure` measure-route (`clusterMeasure`, `AX_cellRecursion`, `omCells`, `descend_size_lt`) is BYPASSED entirely and is never referenced.

This layer fixes the two finite-cardinality oracles on which the whole counting argument rests, plus the coefficient-valuation primitive that feeds the Newton polygon. Throughout, K is a finite extension of p with ring of integers, residue field of size q (a power of the residue characteristic), and n the degree. We model $/\mathfrak{p}^N$ by the *finite ring* (p^N) via p (no measure theory: a box of cardinality $(p^N)^n$). All counts are `Nat.card`, never `Fintype.card`, to dodge missing decidable instances on submodules.

3.1 M0 — ResidueCount: the residual-count oracle

Definition 3.1 (Monic and squarefree-monic count sets). For a finite field F ($Q := F$) and $m : \text{we reuse, verbatim, the } \text{lean}_{uratssets}$

$$\text{LeanUrat.L3.squarefreeMonicDegree } F \ m := \{p \mid p.\text{Monic} \wedge p.\text{natDegree} = m \wedge \text{Squarefree } p\}.$$

Lemma 3.2 (Monic count = Q^m (re-export)). [lean: LeanUrat.OM.M0.card`monicDegree] For a finite field F with $Q = F$ and any m :, $\text{Nat.card}(\text{monicDegree } F \ m) = Q^m$. **NOVEL re-export** of the **CITED** (already-proved, sorry-free) LeanUrat.L3.card_mmonicDegree.

Lemma 3.3 (Squarefree-monic count = $Q^m - Q^{m-1}$ (re-export)). [lean: LeanUrat.OM.M0.card`squarefreeMonicDegree]
For a finite field F , $Q = F$, and $m \geq 2$, $\text{Nat.card}(\text{squarefreeMonicDegree } F \ m) = Q^m - Q^{m-1}$.
NOVEL re-export of the **CITED** (proved, sorry-free) LeanUrat.L3Squarefree.card`squarefreeMonicDegree.

10

Lemma 3.4 (Non-squarefree residual count = Q^{m-1} — the T_{BB3} oracle). [lean: LeanUrat.OM.M0.nonSquarefreeMonicCount] For a finite field F ($Q = F$) and $m \geq 2$,

$$\text{Nat.card } \{R : F[X] \mid R.\text{Monic} \wedge R.\text{natDegree} = m \wedge \neg \text{Squarefree } R\} = Q^{m-1}.$$

This is *exactly* the T_{BB3} residual-equidistribution count: with $Q = q^\delta$, $m = d_S$ it is $(q^\delta)^{d_S-1}$, the descent-triggering (non-separable = non-squarefree, Lemma 3.11(3)) residual-config count of `def : countCellCoeff-ref`. **NOVEL re-export**: the statement and proof already exist as `LeanUrat.MontesAxiom.residualBoxCount`; M0 surfaces it under the oracle name and adds the $(q^\delta)^{d_S-1}$ tie (`residualBoxCounteqfactor`).

Proof sketch. exact LeanUrat.MontesAxiom.residualBoxCount F m hm (lean_{urat}proof : themonicsetsplitsassqfree via an explicit Equiv on Sum, finiteness propagated by `Finite.sumleft/sumright`, and `Nat.cardsumgivesQm = (Qm - Qm-1) + #¬sqfree`; omega with $Q^{m-1} \leq Q^m$). The $(q^\delta)^{d_S-1}$ tie is `residualBoxCounteqfactor F q dS F h d` (hF:F) rewrite then `pushcast`; *ring.Hardpoint* : none; the only subtlety is that $m \geq 2$ is required (for $m \leq 1$ every monic is squarefree, count 0), matching the GMN convention that descent triggers only at residual degree ≥ 2 . $\square$

Lemma 3.5 (Perfectness: non-separable $\Leftrightarrow$ non-squarefree (descent trigger, re-export)). [lean: LeanUrat.OM.M0.not'separable'iff'not'squarefree] Over a finite field F , for $g : F[X]$, $\neg g.\text{Separable} \iff \neg \text{Squarefree } g$. **CITED** (mathlib `PerfectField.separableiffsquarefree`, `PerfectField.ofFinite`, `PerfectField.sep`).

Proof sketch. exact LeanUrat.L3.not_separable_{iff}not_squarefree. (M0 re-export; this is the bridge that lets thenon-squarefreecountof`lem:nonsqfree-count` be read as the descent locus count.) $\square$

Cross-layer reference placeholders. `lem:nonsqfree-count` is consumed at the M8 node-count bijection; `lem:nonsep-iff-nonsqfree` at the M4 descent trigger and the M5 leaf/descend dichotomy. The label `def : countCellCoeff-ref` below names the existing `MontesAxiom.countCellCoeff` so the DAG closes against M7/M8.

Definition 3.6 (Reference: the existing cell coefficient). [lean: LeanUrat.MontesAxiom.countCellCoeff] We name the already-defined `countCellCoeff c q = ((q:)^c.)^(c.dS-1)*L4.bb1Valuec.polygons` so that M0's residual oracle 1

3.2 M1 — QuotientBox: $\mathbb{p}^N$ as the finite ring (p^N)

M1 builds the finite “box” the level- N count lives in. The base field case ($N = 1$, a field) and the genuine finite-ring case ($N \geq 2$) are handled uniformly because everything is phrased through the *semiring*-level `degreeLT` API. (GMN ref: none — this is pure mathlib p -adic/ $\mathbb{Z}\text{Mod}$ plumbing; **CITED** on the mathlib decls, **NOVEL** on the box-count assembly.)

Definition 3.7 (The residue ring and reduction map). [lean: LeanUrat.OM.M1.toZModPow] Fix a rational prime p with `[Fact p.Prime]`. The level- N residue ring is (p^N) and the reduction is the ring hom

$$\text{PadicInt.toZModPow } N : {}_{[p]} \rightarrow^{+*} (p^N).$$

For the present project $=_{[p]}$ (the absolutely-unramified model; the general is a free ${}_{[p]}$ -module and reduces coordinatewise — recorded as a future generalization, not needed for the count). **CITED**: `PadicInt.toZModPow` (mathlib `Mathlib/NumberTheory/Padics/RingHoms.lean`).

Lemma 3.8 (Kernel and surjectivity of reduction). [lean: LeanUrat.OM.M1.toZModPow'surjective]

$$1. \ker(\text{toZModPow } N) = \text{Ideal.span } \{(p : {}_{[p]})^N\} = \mathfrak{p}^N \text{ (CITED: } \text{PadicInt.ker}_{toZModPow}).\text{toZModPow } N \text{ is surjective)}$$

2. The induced map $_{[p]}/\mathfrak{p}^N \simeq^{+*} (p^N)$ is a ring isomorphism (quotientKerEquiv).

NOVEL (the surjectivity/equiv packaging) over **CITED** mathlib lemmas.

Proof sketch. (1) is `PadicInt.kertoZModPowN` *verbatim*. (2) : `for a:, toZModPowN (a :[p]) = (a : (pN))` because both sides are `mapnatCastoftheringhom; mathlib's ZMod.natCastzmod surjective (needs [NeZero(pN)], a 1) gives every $z : (p^N)$ as $(a : \cdot)$, so $z = \text{toZModPowN } (a :_{[p]})$ is in the range. (Alternative: PadicInt.denseRangenatCastpluscontinuity, but then atCast factoring is shorter.) (3) : feed(2) into RingHom.quotient — exactly the pattern mathlib uses for PadicInt.residueField. Hard point : the [NeZero(pN)] instance; supply NeZero`

Lemma 3.9 (Residue-ring cardinality = p^N). [lean: LeanUrat.OM.M1.card' residueRing] $\text{Nat.card } ((p^N)) = p^N$. **CITED** (mathlib `Nat.cardzmod`).

Proof sketch. `exact Nat.cardzmod(pN)(mathlib : Nat.cardzmod(n :) : Nat.card(ZMod n) = n, proved by Nat.cardzmod ZMod.card for  $n \geq 1$ ).` No work. $\square$

Definition 3.10 (The level- N monic box). [lean: LeanUrat.OM.M1.monicBox] For $N, n : \text{set}$

$$\text{monicBox } N \ n := \{p : ((p^N))[X] \mid p.\text{Monic} \wedge p.\text{natDegree} = n\}.$$

This is the finite set of degree- n monic polynomials over the residue ring — the “box” whose cardinality is the denominator of the density. **NOVEL** (mirrors `L3.monicDegree` but over a finite ring (p^N) , not a field). Finiteness: `lem:finite-monicBox`.

Lemma 3.11 (Free-module count of degreeLT over a finite ring). [lean: LeanUrat.OM.M1.card' degreeLT'ring] For any finite commutative ring R (in particular $R = (p^N)$) and $m :$,

$$\text{Nat.card } (\text{degreeLT } R \ m) = (\text{Nat.card } R)^m.$$

NOVEL. This is the finite-ring analogue of `L3.carddegreeLT; the field-only Module.natCardeqpow finrank cannot risk note`, *sower outethrough the line are equivalence degreeLTEquiv and Nat.card_{fun}*.

Proof sketch. The map `Polynomial.degreeLTEquiv R m : degreeLT R m [R] (Fin m → R)` is an R -linear *equivalence* **available over any [Semiring R]** (confirmed: its definition in `Mathlib/RingTheory/Polynomial/Basic.lean` requires only `[Semiring R]`). Take its underlying `Equiv (.toEquiv)` and transport: `Nat.cardcongr(degreeLTEquiv R m).toEquiv, then Nat.cardfun(mathlib Nat.cardfun[Finite Fin m, β = R, using Nat.card (Fin m) = m (Nat.cardeq fin type card, Fintype.cardfin). Finiteness of R(Finite(ZM 1) makes degreeLT R m finite. Hard point: this is the lemma that sidesteps the field requirement; see the build-risk note.` $\square$

Lemma 3.12 (Monic-box cardinality = $(p^N)^n$). [lean: LeanUrat.OM.M1.card' monicBox] $\text{Nat.card } (\text{monicBox } N \ n) = (p^N)^n$, i.e. the box has $(p^N)^n$ elements. **NOVEL.** This is the box count appearing as the density denominator q^{nN} with $q = p$ (the unramified case $(/p) = p$); it carries *no* Haar measure.

Proof sketch. Run the L3 monic-bijection argument over the ring $R = (p^N)$: the map $g \mapsto X^n + g$ is a bijection `degreeLT R n → monicBox N n`. Crucially the L3 proof of *this bijection* uses only ring lemmas valid over any commutative ring: `monicXpow, Monic.addof left with degreeXpow, degreeaddeq rightof degreeXpow` $(p^N)^n$. Hard point: the L3 proof is field-phrased but *field-free*; this lemma re-runs it over R rather than reusing `L3.cardm monicDegree (which is stated for [Field F])`. *I fre-running is awkward, an alternative is to generate*

Lemma 3.13 (Finiteness of the monic box). [lean: LeanUrat.OM.M1.instFiniteMonicBox] `Finite (monicBox N n)`. **NOVEL** (ring-analogue of `L3.squarefree.instFiniteMonicDegree`).

Proof sketch. `Finite.ofnjective into degreeLT R (n + 1) sending p ↦ ⟨p, ...⟩` (a monic of `natDegree n` has degree $< n+1$ via `degreeeq natDegree and Nat.ltsucc self`); `degreeLT R (n + 1)` is finite since R is finite and it is a *generalized*. $\square$

BUILD-RISK (field vs. finite ring) and the stated fallback. `lem:card-degreeLT-ring` is the load-bearing risk: L3 obtains $\text{Nat.card}(\text{degreeLT } F m) = Q^m$ via `Module.natCard_eq_pow_finrank`, but that lemma is for $N \geq 2$ (a finite *ring*, not a field). The fallback, recorded as a separate lemma variant, is to count directly through the linear *equivalence* `Polynomial.degreeLTEquiv R m : degreeLT R m [R] (Fin m → R)` (valid for any `[Semiring R]`) and `Nat.card_fun`, *never invoking finrank or a division – ring instance. We state the fallback as its own node so the formalization can pick it unconditionally :*

Lemma 3.14 (Fallback: degreeLT count via degreeLTEquiv + Nat.card_fun). `[lean: LeanUrat.OM.M1.card' degreeLT' via 'fun` For `[CommRing R] [Finite R]` and $m :$, $\text{Nat.card}(\text{degreeLT } R m) = (\text{Nat.card } R)^m$, proved *without* `Module.natCard_eq_pow_finrank` or any field hypothesis : `Nat.card_congr (Polynomial.degreeLTEquiv R m).toEquiv`, *ring route are visibly distinct nodes, and* **lem:card-degreeLT-ring** *is exact card_degreeLT_via_fun*. **NOVEL; CITED**

Proof sketch. As in `lem:card-degreeLT-ring`. The only mathlib facts used — `degreeLTEquiv ([Semiring R])` and `Nat.card_fun ([Finite])` — both hold for a finite commutative ring, so the fallback is unconditional a field, both routes agree and equal `L3.card_degreeLT`.) $\square$

3.3 M2 — PadicValuation: coefficient valuations and the support Finset

M2 supplies the integer-valued coefficient valuations $v_i(f) = v((f).\text{coeff } i)$ and packages the support as a Finset of lattice points (i, v_i) — the raw input to the Newton polygon of M3. No geometry and no measure here. (GMN ref: the v_i are the data of the φ -adic expansion; the order-1 polygon is built on them, Def. before 1.21. **CITED** on `PadicInt.valuation`, **NOVEL** on the support assembly.)

Definition 3.15 (Coefficient valuation). `[lean: LeanUrat.OM.M2.coeffVal]` For $f :_{[p]} [X]$ and $i :$,

$$\text{coeffVal } f \ i := (f.\text{coeff } i).\text{valuation} \ ,$$

where `PadicInt.valuation : [p]BisMathlib's normalized p-adic valuation (valuation x = (x :_{[p]}).valuation.toNat)`, *with coefficients that are 0 the valuation is reported as 0 but the point is excluded from the support (def: val-support); the* is encoded by absence from the Finset, which is what the lower Newton polygon needs.

Lemma 3.16 (Valuation arithmetic facts re-exported). `[lean: LeanUrat.OM.M2.coeffVal'facts]` For $f :_{[p]} [X]$: (i) `coeffVal f i = 0` whenever `f.coeff i` is a unit (in particular for a monic top coefficient = 1, via `valuation_one`); (ii) *multiplicativity* $v(xy) = v(x) + v(y)$ on non-zero x, y (`PadicInt.valuation_mul`); (iii) $v(p^k c) = k + v(c)$ (`PadicInt.valuation_pow_mul`); (iv) *the ultrametric* $\min(v(x), v(y)) \leq v(x + y)$ for $x + y \neq 0$ (`PadicInt.valuation_add`). **CITED** *re-export; these are the lemmas M3 needs to prove convexity/lower-minorant of the Newton polygon. No new proof content (each is a one-line application of the named mathlib lemma).*

Proof sketch. Each clause exacts the named `PadicInt` lemma after unfolding `coeffVal`; (i) is `valuation_one / isUnit  $\Rightarrow$  no = 1  $\Rightarrow$  valuation 0`. Bundled as a single structure/conjunction so M3 imports one node. $\square$

Definition 3.17 (The valuation support Finset). `[lean: LeanUrat.OM.M2.valSupport]` For $f :_{[p]} [X]$ define

$$\text{valSupport } f := (f.\text{support}).\text{image } (i \mapsto (i, \text{coeffVal } f \ i)) : \text{Finset } (\times),$$

using mathlib's `Polynomial.support : R[X] → Finset` (the finite set of i with `coeff i` $\neq 0$) and `Finset.image`. The points are exactly the (i, v_i) with $a_i \neq 0$ — the dots whose lower convex hull is the Newton polygon of M3. **NOVEL** (assembly), **CITED** on `Polynomial.support / Finset.image`.

Lemma 3.18 (Support membership / nonemptiness for monic f). [lean: LeanUrat.OM.M2.valSupport`facts] For $f :_{[p]} [X]$: (i) $(i, v) \in \text{valSupport } f \iff f.\text{coeff } i \neq 0 \wedge v = \text{coeffVal } f \ i$ ($\text{Finset.mem}_i \text{image} + \text{Polynomial.mem}_s \text{support}_i f f$); (ii) *iff is monic of natDegree then* $(n, 0) \in \text{valSupport } f$ (top coeff = 1, valuation 0 by `lem:val-facts(i)`), so the support is nonempty and pins the right endpoint of the polygon at height 0. **NOVEL**; elementary Finset reasoning.

Proof sketch. (i) `simp [valSupport, Finset.mem_image, Polynomial.mem_ssupport_i f f]`. (ii) *the top coefficient of a monic* 0 puts `natDegree` in support, and `coeffVal f n = valuation 1 = 0` (`PadicInt.valuation_one`). *Hardpoint* : *none; this fixes the polygon's right vertex, which M3 relies on for the lower-minorant to be well-defined (finite, and*

Layer 0 hand-off. M0 exposes the residual-count oracle (`lem:nonsqfree-count`, the T_{BB3} value, = first factor of `countCellCoeff`) and the descent-trigger equivalence (`lem:nonsep-iff-nonsqfree`); M1 exposes the finite box and its count $(p^N)^n$ (`lem:card-monicBox`, the density denominator) plus the quotient iso (`lem:ker-surj`); M2 exposes the coefficient valuations and the support Finset (`def:val-support`) feeding M3's Newton polygon. The single carried build-risk is `lem:card-degreeLT-ring's` field-vs-ring issue, dispatched by the stated fallback `lem:card-degreeLT-fallback`.

4 Layer 1: Newton polygons and residual polynomials (the new geometry)

This layer builds the two genuinely-new geometric objects of the GMN machinery as concrete combinatorial data over $\mathcal{O}_K$ = (we work with K = throughout the formalization core; the residue tower of higher OM orders is supplied later by M5). Module M3 (`LeanUrat.OM.M3.NewtonPolygon`) constructs the lower Newton polygon of a monic $f \in [X]$ as an explicit lower convex minorant of the coefficient-valuation points $(i, v_i(f))$, extracts its vertices, sides, and rational slopes, and *bridges* the resulting lattice data to the already-proved `LeanUrat.L4.LatticePolygon` consumed by the BB1 box count. Module M4 (`LeanUrat.OM.M4.ResidualPolynomial`) defines the residual polynomial R_S attached to a side S over the finite residue field, proves the degree formula $\deg R_S = \ell(S)/e$, that its leading and trailing coefficients are nonzero, and that over the (perfect, because finite) residue field separability of R_S coincides with squarefreeness — making the GMN descent trigger (Lemma 3.11(3)) a decidable, mathlib-native predicate.

Layer-wide conventions. p is the rational prime, the p -adic integers (`_[p]` in mathlib), and $v = \text{PadicInt.valuation} : \rightarrow \mathbb{N}$ the p -adic valuation *normalized to* $v(p) = 1$ (`mathlib.PadicInt.valuation_p`). We write $f = X^n + \sum_{i < n} a_i X^i$, $v_i := v(a_i)$, q the residue cardinality. We emphasise one Lean hazard that recurs below: `PadicInt.valuation` returns a *natural number* obtained as $(x : \text{PadicInt}).\text{valuation.toNat}$, and `PadicInt.valuation_zero` gives $v(0) = 0 = v(1)$; hence v alone does *not* detect $a_i = 0$, and every height/support construction below must guard on the predicate `a_i ≠ 0` explicitly rather than reading it off the valuation.

4.1 M3: the lower Newton polygon

Definition 4.1 (Coefficient-valuation support). [lean: LeanUrat.OM.M3.npSupport] For a monic $f \in [X]$ of `natDegree` n , the *Newton support* is the finite set of plotted lattice points

$$\text{supp}(f) := \{ (i, v_i(f)) : i \in \{0, \dots, n\}, f.\text{coeff } i \neq 0 \} \subseteq \mathbb{N} \times \mathbb{N},$$

realized in Lean as a `Finset ( × )` obtained by filtering `Finset.range (n+1)` through the *coefficient-nonzero* guard and mapping $i \mapsto (i, (f.\text{coeff } i).\text{valuation})$. The endpoint $(n, 0)$ is always present (monic $\Rightarrow f.\text{coeff } n = 1, v = 0$).

Flag: NOVEL (GMN Def 1.3/§1.1, the φ -Newton polygon at order 1). The guard on `coeff i` $\neq 0$ is mandatory: without it the spurious point $(i, 0)$ from a zero coefficient would corrupt the lower hull. We carry the index set as `Finset.range (n+1)` rather than `Fin(n+1)` so that the bridge index $i \in \{0, \dots, e-1\}$ to `L4.LatticePolygon` (width e) is a sub-range, not a coerced `Fin`.

Definition 4.2 (Slope-indexed lower height). [`lean: LeanUrat.OM.M3.npHeight`] The *lower Newton height* of f at abscissa $x \in \mathbb{Q}$ (we only evaluate at integer x , but the value is rational) is the largest lower bound consistent with all plotted points lying on or above a line through them:

$$\text{npHeight}_f(x) := \min_{(i,v) \in \text{supp}(f)} \left[v + \lambda_x^*(i, v) \cdot (x - i) \right] \quad \text{realized as} \quad \inf_S (b_S + m_S x),$$

where the infimum ranges over the finite set of *candidate supporting lines* $y = m_S x + b_S$ determined by each ordered pair of support points $\{(i, v_i), (j, v_j)\}$, $i < j$, with slope $m_S = (v_j - v_i)/(j - i) \in \mathbb{Q}$ and intercept $b_S = v_i - m_S i$. Concretely `npHeight` is a `Finset.inf'` (the support is nonempty: it contains $(n, 0)$) over $\mathbb{Q}$ with its conditionally-complete lattice order, of the affine evaluations $\lambda \mapsto (\text{line through the pair})$ at the queried x .

Flag: NOVEL. Why not mathlib convexHull. `mathlib's convexHull s (Mathlib.Analysis.Convex.Hull, ClosureOperator.ofCompletePred (Convex ))` is defined for a module E over an *ordered field/division ring* $\mathbb{T}$; it computes the set-theoretic convex hull as a subset of a $\mathbb{T}$ -vector space and gives no handle on the *lower* boundary, on vertices as a finite list, or on rational slopes as data. The Newton polygon lives over the lattice $\mathbb{N} \times \mathbb{N}$ (no $\mathbb{T}$ -module structure of the right kind), and we need the *lower hull only*, with its breakpoints and slopes as extractable combinatorial data feeding M4/L4. We therefore build the lower minorant *directly* as a finite `inf'` of affine functions — the standard “min over candidate supporting slopes” formulation — which is both definitionally computable and avoids importing the analytic convexity hierarchy. This is the design decision the prompt anticipates.

Lemma 4.3 (Lower minorant property). [`lean: LeanUrat.OM.M3.npHeight'le`] For every plotted point $(i, v_i) \in \text{supp}(f)$ we have $\text{npHeight}_f(i) \leq v_i$; i.e. the polygon lies weakly below every coefficient-valuation point.

Proof sketch. By `Finset.inf'_le`: the candidate line through the pair $\{(i, v_i), (i, v_i)\}$ (degenerate, or any pair containing (i, v_i)) evaluates to exactly v_i at $x = i$, so the infimum at $x = i$ is $\leq v_i$. Formally we take the line determined by the pair $((i, v_i), (n, 0))$ if $i < n$, or the endpoint line for $i = n$; `Finset.inf'_le` applied to that index discharges the bound. The only subtlety is bundling the candidate-line family as a `Finset` indexed by ordered pairs and proving it nonempty (contains the endpoint), which follows from $(n, 0) \in \text{supp}(f)$ via monicity (`Polynomial.Monic.coeff_natDegree, = 1` $\neq 0$). $\square$

Lemma 4.4 (Convexity / lower-hull breakpoints). [`lean: LeanUrat.OM.M3.npHeight'convexOn`] npHeight_f is a convex piecewise-linear function on $[0, n]$: a finite infimum of affine functions is concave in general, but the *lower* Newton minorant taken as the greatest convex minorant of the support is convex; equivalently, the successive side-slopes are strictly increasing. We record this as: the slopes $\lambda_1 < \lambda_2 < \dots < \lambda_g$ of the sides, read left to right, are strictly increasing.

Proof sketch. We do *not* obtain convexity from `Finset.inf'` directly (an inf of affine maps is concave); instead we *define* the polygon as the lower convex hull and prove the breakpoint characterization. Practically, `npHeight` is computed by the standard incremental lower-hull scan over

$\text{supp}(f)$ sorted by abscissa: a point is a vertex iff it is not above the segment joining its hull neighbours, the cross-product test $(j-i)(w-v_i) \leq (k-i)(v_j-v_i)$ on integer coordinates (no division, avoids $\mathbb{Q}$ rounding). Strict slope increase between consecutive sides is then the hull-scan invariant, proved by induction on the scan. **Hard point:** the cleanest Lean route is to make `npHeight` the value of an explicitly-constructed vertex list `npVertices` (Definition 4.5) and prove the minorant property (Lemma 4.3) and convexity *from* that list, rather than from an `inf`’; the `inf`’ form of Definition 4.2 is then a *theorem* (`npHeight_eq_inf`’) equating the two, used only where the variational form is convenient. We flag this as the main design fork in M3. $\square$

Definition 4.5 (Vertices, sides, slopes). [lean: LeanUrat.OM.M3.npVertices] The *vertex list* `npVertices(f) : List ( × )` is the lower-hull breakpoint sequence, sorted by abscissa, computed by the integer cross-product hull scan over $\text{supp}(f)$. A *side* S is a consecutive pair of vertices $((i, v_i), (j, v_j))$ with $i < j$; its *length* (horizontal) is $\ell(S) := j - i \in \mathbb{N}$, its *slope* $\lambda_S := -(v_j - v_i)/(j - i) \in \mathbb{Q}$ (the GMN sign convention: descending sides of the lower hull have $\lambda_S \leq 0$), written in lowest terms as $\lambda_S = -h_S/e_S$ with $e_S = (j-i)/\text{Nat.gcd} \dots$ the *ramification denominator* and h_S the numerator, $\text{gcd}(h_S, e_S) = 1$. We package `npSides(f) : List Side` where each `Side` carries (i, j, v_i, v_j) and the derived (ℓ, e, h, λ) .

Flag: NOVEL.

Lemma 4.6 (Finiteness and rationality). [lean: LeanUrat.OM.M3.npVertices'finite] `npVertices(f)` and `npSides(f)` are finite lists of length $\leq n+1$ resp. $\leq n$, and every slope λ_S is rational with denominator $e_S \mid \ell(S) \leq n$.

Proof sketch. Finiteness is immediate: the vertex list is a sublist of a sort of $\text{supp}(f)$, itself a filter of `Finset.range (n+1)`, so length $\leq n+1$ (`List.length_le` via `List.Sublist.length_le`). Sides number one fewer than vertices. Rationality of λ_S is by construction ($\lambda_S \in \mathbb{Q}$ as a quotient of integers); the denominator bound $e_S \mid \ell(S)$ is `Rat.den` of a reduced fraction with $\ell(S)$ in the denominator before reduction, hence `Nat.div_dvd_of_dvd / Rat.num_div_den`; $\ell(S) \leq n$ since the total horizontal width is n (sum of side lengths telescopes to $n-0 = n$ via `List.sum` over consecutive abscissa gaps). $\square$

Definition 4.7 (Bridge to the L4 lattice polygon). [lean: LeanUrat.OM.M3.toLatticePolygon] `toLatticePolygon(f) : L4.LatticePolygon` is built from `npVertices(f)` by:

- `width := n` (the polygon lives on $[0, n]$; for an order- r refinement the relevant width is the residual side length e , supplied at the call site — here we expose the full-degree polygon and let M7 restrict to a side);
- `ceilHeights i := ⌈npHeightf(i)⌉` for $i \in \{0, \dots, n-1\}$, computed as `(npHeight f i).ceil.toNat` (the closed BB1-A column exponent $A = \sum_i \lceil h_i \rceil$);
- `isVertex i := (i ∈ (npVertices f).map Prod.fst)`, a `Bool`.

Flag: NOVEL (the bridge); the target type `L4.LatticePolygon` and its consumers `L4.newtonExponent`, `L4.newtonVertexCount`, `L4.bb1Value`, `L4.columnMeasure`, `L4.cellVolume_eq` are CITED (already proved in `LeanUrat.L4`).

Lemma 4.8 (Bridge correctness). [lean: LeanUrat.OM.M3.toLatticePolygon'spec] For the polygon $\Pi = \text{toLatticePolygon } f$: (a) `L4.newtonVertexCount` $\Pi = \#\{\text{vertices of } \text{npVertices}(f) \text{ with abscissa } < n\}$; (b) `L4.newtonExponent` $\Pi = \sum_{i < n} \lceil \text{npHeight}_f(i) \rceil = A(\Pi)$; (c) at a vertex abscissa i the height $\text{npHeight}_f(i)$ is an *integer*, so $\lceil h_i \rceil = h_i$. Consequently `L4.bb1Value` ΠQ is the genuine Newton-box value $(1 - Q^{-1})^V Q^{-A}$ for this f .

Proof sketch. (a),(b) unfold `L4.newtonVertexCount/newtonExponent` (`Finset.card/Finset.sum` over `Fin n`) and rewrite the `isVertex/ceilHeights` fields by their definitions; the vertex-count is a `Finset.filter` cardinality matching the abscissa-membership `Bool`. (c) is the key arithmetic content: a hull breakpoint (i, v_i) has $\text{npHeight}_f(i) = v_i \in \mathbb{N}$ because the two incident sides both pass through the integer point (i, v_i) ; formally the `inf'` (Lemma via `npHeight_eq_inf'`) is attained at a line through (i, v_i) , giving value v_i , and the minorant bound (Lemma 4.3) gives $\geq$ is impossible, so equality. This is exactly the hypothesis `L4.cellVolume_eq` implicitly uses ($\lceil h_i \rceil = h_i$ at vertices collapses the product). **Hard point:** matching `Fin n` (L4's index type) against `Finset.range n` (M3's): a `Fin.sum_univ_eq_sum_range/Finset.sum_range` re-indexing, mechanical but fiddly. $\square$

4.2 M4: the residual polynomial of a side

Fix a monic $f \in [X]$ and a side $S \in \text{npSides}(f)$ of slope $\lambda_S = -h_S/e_S$ (lowest terms, Definition 4.5). Let k_S be the residue field; in the order-1 core $k_S == /p = \mathbb{Z} \bmod p$ (a *finite field*); at higher orders M5 supplies the finite tower field F_r . The residual is computed from the coefficients lying *exactly* on the side.

Definition 4.9 (On-side coefficients / residual coefficient). [lean: LeanUrat.OM.M4.residualCoeff] With $S = ((i_0, v_0), (j_0, v_1))$, abscissae of *lattice points on S* are $i_0, i_0 + e_S, i_0 + 2e_S, \dots, i_0 + \ell(S)$ (only multiples of e_S can be integral points on a slope- $-h_S/e_S$ line through an integer point). For the t -th such abscissa $i_t = i_0 + te_S$ ($0 \leq t \leq d$, $d := \ell(S)/e_S$), the *residual coefficient* $c_t \in k_S$ is the image in k_S of the unit part of $a_{i_t}/p^{v_{i_t}}$ *provided* (i_t, v_{i_t}) *lies on S* (i.e. $v_{i_t} = v_0 + (i_t - i_0) \cdot (\text{integer height step})$), and $c_t := 0$ otherwise (the point lies strictly above S). Formally $c_t := \text{PadicInt.toZMod}(u_{i_t})$ where $u_{i_t} = a_{i_t} \cdot p^{-v_{i_t}}$ is the `PadicInt.unitCoeff`, lifted through the residue map.

Flag: CITED (GMN Def 2.20, residual coefficient) / NOVEL (the Lean encoding).

Definition 4.10 (Residual polynomial of a side). [lean: LeanUrat.OM.M4.residualPoly] The *residual polynomial* of the side S is

$$R_S(f)(Y) := \sum_{t=0}^d c_t Y^t \in k_S[Y], \quad d = \ell(S)/e_S,$$

the polynomial whose Y^t -coefficient is the residual coefficient c_t of Definition 4.9 (GMN Def 2.21). In Lean: `t Finset.range (d+1), Polynomial.C (residualCoeff f S t) * Polynomial.X t`.

Flag: CITED (GMN Def 2.21, the order-r residual polynomial over the finite F_r).

Lemma 4.11 (Endpoint coefficients are units). [lean: LeanUrat.OM.M4.residualCoeff'endpoints'ne'zero] The endpoints of the side are vertices of the polygon, so (i_0, v_0) and (j_0, v_1) lie exactly on S ; hence the residual coefficients c_0 and c_d are images of *units* of , and $c_0 \neq 0$, $c_d \neq 0$ in k_S .

Proof sketch. (i_0, v_0) is a vertex (left endpoint of S), so $a_{i_0} \neq 0$ (it is a support point) and $v_{i_0} = v_0$; its unit part $u_{i_0} = a_{i_0}p^{-v_0}$ has $v(u_{i_0}) = 0$, i.e. u_{i_0} is a -unit (`PadicInt.isUnit_iff / PadicInt.unitCoeff_spec`). The residue map `PadicInt.toZMod` sends units to nonzero elements of the residue field (kernel is the maximal ideal p , `PadicInt.ker_toZMod`); so $c_0 = \overline{u_{i_0}} \neq 0$. Same for c_d at the right endpoint (j_0, v_1) . This is GMN's " c_s and c_{s+de} are always nonzero". $\square$

Lemma 4.12 (Degree formula). [lean: LeanUrat.OM.M4.residualPoly'natDegree] $\deg R_S(f) = d = \ell(S)/e_S$, and $R_S(f)$ is never divisible by Y (its constant term $c_0 \neq 0$).

Proof sketch. Upper bound: R_S is a sum over $\text{range}(d+1)$, so $\deg \leq d$ (`Polynomial.natDegree_sum_le / natDegree_C_mul_X_pow_le`). Lower bound: the Y^d -coefficient is $c_d \neq 0$ by Lemma 4.11; thus $\text{coeff } d R_S = c_d \neq 0$ giving $\deg \geq d$ (`Polynomial.le_natDegree_of_ne_zero`). Equality follows. The constant term is $c_0 \neq 0$, so $Y \nmid R_S$ (`Polynomial.X_dvd_iff  $\Leftrightarrow$  coeff 0 = 0`). **Hard point (index arithmetic):** establishing that $\ell(S) = e_S \cdot d$ *exactly* (the side length is a multiple of the ramification denominator) requires the reduced-fraction lemma $e_S = \ell(S) / \gcd(\dots)$ from Definition 4.5 and `Nat.div_mul_cancel` under $e_S \mid \ell(S)$ (Lemma 4.6); off-by-one between “ $d+1$ coefficients” and “degree d ” is the usual `Finset.range` bookkeeping. $\square$

Lemma 4.13 (Residual is monic after normalization). [`lean: LeanUrat.OM.M4.residualPoly`monic`normalized`] The normalized residual $\tilde{R}_S := c_d^{-1} \cdot R_S(f)$ is monic of degree d over k_S (legitimate since $c_d \in k_S^\times$, k_S a field). All downstream squarefree/separable claims are invariant under this unit rescaling.

Proof sketch. $\deg(c_d^{-1} R_S) = \deg R_S = d$ (`Polynomial.natDegree_C_mul` with $c_d^{-1} \neq 0$); its leading coefficient is $c_d^{-1} \cdot c_d = 1$ (`Polynomial.leadingCoeff_C_mul / leadingCoeff`). `Squarefree/Separable` are preserved by multiplication by a unit `C u` (`Squarefree.of_mul_left/IsUnit` associativity; `Polynomial.separable_C_mul_iff`-style), so the descent trigger below may be stated for either R_S or $\tilde{R}_S$. $\square$

Theorem 4.14 (`Separable  $\Leftrightarrow$  squarefree over the residue field`). [`lean: LeanUrat.OM.M4.residual`separable`iff`squarefree`] Let k_S be the finite residue field of the side. Then for $R = R_S(f)$ (equivalently $\tilde{R}_S$),

$$R.\text{Separable} \iff \text{Squarefree } R, \quad \text{and} \quad \neg R.\text{Separable} \iff \neg \text{Squarefree } R.$$

Proof sketch. k_S is finite, hence perfect: the instance `PerfectField.ofFinite` (`Mathlib.FieldTheory.Perfect`, requires `[Finite k_S]`) supplies `[PerfectField k_S]`. Then `PerfectField.separable_iff_squarefree` gives the biconditional verbatim, and `not_congr` the negated form. This is exactly the already-proved `L3.separable_iff_squarefree / L3.not_separable_iff_not_squarefree` specialized to k_S ; we re-export through M4 so the descent trigger reads off the residual directly. *No new proof content* — the perfectness of every finite residue field at every OM order is the linchpin (GMN fact (I)): there are no inseparable-irreducible exceptions in any characteristic. $\square$

Flag: CITED (`mathlib PerfectField.ofFinite, PerfectField.separable_iff_squarefree; reuse of LeanUrat.L3`).

Definition 4.15 (Descent trigger). [`lean: LeanUrat.OM.M4.needsDescent`] A side S *needs descent* iff its residual polynomial is non-separable, equivalently (by Theorem 4.14) non-squarefree, equivalently has a repeated irreducible factor:

$$\text{needsDescent}(f, S) :\iff \neg \text{Squarefree}(R_S(f)).$$

The complementary leaf condition `Squarefree( $R_S(f)$ )` is the GMN “ f -complete at this side” / multiplicity-1 leaf case.

Flag: CITED (`GMN Lemma 3.11(3): descent past order  $r$  fires iff some order- $r$  residual is non-separable; equivalently Cor 1.20/3.8 multiplicity-1  $\Rightarrow$  leaf`). This is the exact predicate M5 branches on (leaf vs. child `OMType`) and the locus M0/M8 count as $(q^\delta)^{d_S-1}$.

Lemma 4.16 (Descent trigger localizes to a repeated factor). [`lean: LeanUrat.OM.M4.needsDescent`iff`exists`repeated`] `needsDescent(f, S)` holds iff there exists a monic irreducible  $\psi \in k_S[Y]$  and  $\mu \geq 2$  with  $\psi^\mu \mid R_S(f)$ . Each such  $(\psi, \mu)$  is the data of one descent child (feeding M5’s `OMCell.Child: factor = ψ , mult = μ , hmult: $2 \leq \mu$, hdvd: $\psi^\mu \mid R_S(f)$).`

Proof sketch. $\neg\text{Squarefree } R \Leftrightarrow \exists$ a non-unit ψ with $\psi^2 \mid R$ (Squarefree unfolds to: every ψ with $\psi^2 \mid R$ is a unit). Over the field k_S , factor ψ into irreducibles ($\text{UniqueFactorizationMonoid}$ on $k_S[Y]$); a non-squarefree R has some irreducible factor of multiplicity ≥ 2 ($\text{multiplicity/Polynomial.roots}$ or $\text{UniqueFactorizationMonoid.squarefree_iff_factorization}$ -style), giving the witness (ψ, μ) with $\mu =$ that multiplicity ≥ 2 and $\psi^\mu \mid R$. Conversely $\psi^2 \mid R$ with ψ a non-unit forces $\neg\text{Squarefree}$. **Hard point:** extracting a *maximal* μ (the exact multiplicity) rather than merely ≥ 2 needs $\text{multiplicity/emultiplicity}$ finiteness on $k_S[Y]$ ($R \neq 0$ since $c_d \neq 0$); this is the sizable new bookkeeping that M5 will consume to set the child cluster size $= \mu$. $\square$

Layer-1 open risks / hard points (summary). (1) The convexity of npHeight (Lemma 4.4) is genuinely the only analytically-flavoured node; the recommended route is to make the explicit *vertex list* (integer cross-product hull scan, Definition 4.5) primitive and derive both the minorant and the inf' form from it, sidestepping any inf -of-affine concavity clash. (2) The index arithmetic $\ell(S) = e_S \cdot d$ and the $\text{Fin } n$ vs. $\text{rangen } n$ re-indexing across the M3 $\rightarrow$ L4 bridge (Lemmas 4.12, 4.8) is mechanical but error-prone. (3) The $\text{PadicInt.unitCoeff/toZMod}$ plumbing for the residual coefficients (Definition 4.9, Lemma 4.11) is new and must guard $\text{coeff } i \neq 0$ explicitly because $\text{PadicInt.valuation}$ cannot distinguish $a_i = 0$ from a unit ($v(0) = v(1) = 0$). (4) Exact-multiplicity extraction (Lemma 4.16) via multiplicity on $k_S[Y]$ is the new piece M5 depends on. The separability $\Leftrightarrow$ squarefreeness core (Theorem 4.14) is essentially free via $\text{mathlib} + \text{L3}$.

5 Layer 2: the recursive OM type and its termination (M5, M6)

This layer builds the recursive Okutsu–Montes (OM) datatype on top of the Newton-polygon (M3) and residual-polynomial (M4) machinery of Layer 1, and proves that the descent it triggers is well-founded with depth bounded by the degree n . Two MontesData obligations are discharged here: cells.descend and $\text{finiteTermination}(\text{thm:cells-descend-discharge}, \text{thm:finite-termination-discharge})$.

Throughout, K is a finite extension of p , its valuation ring, $q = |q|$ the residue cardinality, n the degree, and $f = X^n + \sum_i a_i X^i \in [X]$ monic. An OM type of order r carries a tower of *finite* residue fields $0 \subseteq_1 \subseteq \dots \subseteq_r$, a key polynomial ϕ_r as imported data (GMN Thm 2.11), an order- r lattice Newton polygon N_r (reusing L4.LatticePolygon), and an order- r residual $R_r \in_r [Y]$ (M4, GMN Def 2.21).

We rely on the following Layer-1 nodes (defined in M3/M4): the lattice Newton polygon $\text{def:newton-polygon/L4.LatticePolygon}$ ($\text{def:lattice-polygon-bridge}$); the order- r residual polynomial def:residual-poly with its degree formula $\text{lem:residual-degree}$ ($\deg R_S = \ell(S)/e$) and its separability characterisation over the finite (hence perfect) residue field, $\text{lem:residual-sep-iff-sqfree}$ (R_S separable $\iff R_S$ squarefree, via $\text{PerfectField.separable_iff_squarefree}$ and $\text{PerfectField.ofFinite}$). We also reference, as the *count shadow* we shadow with real residual data, the abstract $\text{PadicMeasure.OMShape/OMC}$ of the dead measure route ($\text{ax:om-shape-shadow}$), which is bypassed but whose field layout we mirror.

5.1 M5: the order- r OM type

Definition 5.1 (The residue-field tower as an algebra chain). [lean: $\text{LeanUrat.OM.M5.ResidueTower}$] A *finite residue tower* of length r is the data of a sequence of fields $0, \dots, r$ together with, for each $0 \leq i < r$, an $_i$ -algebra structure on $_{i+1}$, such that every $_i$ is a finite field; in Lean this is the OMShape -style bundle

$$\text{Field } _i, \quad \text{Finite } _i, \quad \text{Algebra } _i \text{ } _{i+1},$$

carried as instance fields (cf. `PadicMeasure.OMShape.{instField,instFinite,instBaseField,instAlg}`). The composite degree is $[r:0] = \prod_{i < r} [i+1:i] = f_0 f_1 \cdots f_{r-1}$, computed by `Module.finrank_mul_finrank` along the tower. The single *base/top* pair actually consumed downstream (mirroring `OMShape`'s `baseField/Fr`) is $0 =_q$ and r , with a genuine `Algebra 0 r` and $\delta := [r:0] = \text{Module.finrank } 0 r$. **[CITED, GMN §2.1 tower; fact (I).]**

Lemma 5.2 (Residue-field cardinality is a q -power). `[lean: LeanUrat.OM.M5.card'Fr]` For a residue tower with base 0 of cardinality q and $\delta = [r:0]$, $\text{Nat.card } r = q^\delta$.

Proof sketch. Verbatim port of `PadicMeasure.natCardFr`. From `Finite r` and the algebra chain, `Module.Finite 0 r` holds by `Module.Finite.of_finite`. Then `Module.natCard.eq_pow_finrank` gives $\text{Nat.card } r = (\text{Nat.card } 0)^{\text{finrank } 0 r} = q^\delta$, rewriting δ by its definition $\delta = \text{finrank } 0 r$ (the A0 tie). **[NOVEL, trivial port.]** $\square$

Definition 5.3 (Order- r OM type). `[lean: LeanUrat.OM.M5.OMType]` An *OM type of order r* is a (non-recursive at the record level; recursion is carried by the cell children, see `def:om-cell`) structure

$$\text{OMType} := \left(\underbrace{r}_{\text{order}}, \underbrace{(0 \subseteq \cdots \subseteq r)}_{\text{tower, def:residue-tower}}, \underbrace{\phi_r}_{\text{key poly}}, \underbrace{N_r : \text{L4.LatticePolygon}}_{\text{order-}r \text{ Newton polygon}}, \underbrace{R_r :_r [Y]}_{\text{residual}}, \underbrace{\delta, d_r}_{\text{degrees}}, \underbrace{cs}_{\text{cluster size}} \right)$$

with the *A0 ties* enforced as proof fields (exactly the `OMShape` discipline, `ax:om-shape-shadow`):

- $\delta = \text{Module.finrank } 0 r$ (`hResidueDeg`), so δ is a genuine extension degree, not a free integer;
- $d_r = R_r.\text{natDegree}$ (`hResidualDeg`), so the residual degree is the degree of the *actual* residual;
- ϕ_r is imported *data* of degree $m_r = m_{r-1}e_{r-1}f_{r-1}$ (GMN Thm 2.11, Def 2.12): **only its degree m_r and the lifted residual R_r are used**; it is *never reconstructed* from the lower data;
- N_r is the order- r lattice (principal) polygon, reusing `L4.LatticePolygon` verbatim (M3 bridge `def:lattice-polygon-bridge`; GMN Def 2.3/1.1, fact (II)).

The *cluster size* is $cs(T) := cs$, the well-founded descent measure; by the child law (`def:om-cell-child`) it equals the residual multiplicity at a descent. **[CITED for ϕ_r, N_r, R_r (GMN Thm 2.11/Def 2.3/Def 2.21); NOVEL for the Lean bundling and the A0 ties, ported from `OMShape`.]**

Remark 5.4 (Relation to the count shadow `OMShape`). `OMType` is a refinement of the abstract `PadicMeasure.OMShape` (`ax:om-shape-shadow`): both carry `(size, order, r, 0, residueDeg, N_r, residual, residualDeg)` with the *same* two A0 ties (`residueDeg = finrank`, `residualDeg = residual.natDegree`). The difference is provenance: in `OMShape` the residual is an opaque carried field for the *measure* route (now dead), whereas `OMType`'s R_r is the *real* order- r residual produced by M4 (`def:residual-poly`) from the Newton side of N_r . The eventual `decode : ClusterShape → OMType` of M9 plays the role `OMShape` played in the `Interface.OMBridge` (D1), but now with honest residual data; `PadicMeasure.*` is referenced only to note it is bypassed. **[NOVEL bookkeeping.]**

Definition 5.5 (Cluster size = residual multiplicity). `[lean: LeanUrat.OM.M5.clusterSize]` For an `OMType` T the cluster size is the degree-of-cluster measure $cs(T) \in$. At the *root* of a freshly-formed type it is initialised from the parent residual: when T descends along a multiplicity- μ irreducible residual factor $P \mid R_{\text{parent}}$ (a repeated factor, $\mu \geq 2$), the child's cluster size is $cs(T_{\text{child}}) = \mu$ (GMN

child-size law; mirrored from `OMCell.Child.hsize : node.size = mult`). The multiplicity μ is `Multiset.count P (UniqueFactorizationMonoid.factors R)`, equivalently `Polynomial.rootMultiplicity` of the linear factor when $P = Y - \alpha$. [CITED, GMN Cor 3.8 child-size; NOVEL Lean tie.]

Definition 5.6 (OM cell child: the descent witness). [lean: `LeanUrat.OM.M5.OMCell.Child`] A *descent child* of T is the bundle (the count shadow’s `OMCell.Child` with real residuals)

$$\text{Child } T := (\text{node} : \text{OMType}, P :_r [Y], \mu :, h_\mu : 2 \leq \mu, h_l : P^\mu \mid R_r, h_{cs} : cs(\text{node}) = \mu),$$

encoding GMN Lemma 3.11(3) *in the data*: descent fires *only* at a multiplicity- ≥ 2 (non-separable) factor of the **actual** residual R_r , and the child’s cluster size is that multiplicity. By `lem:residual-sep-iff-sqfree` $\exists$ such a (P, μ) with $\mu \geq 2$ iff R_r is non-squarefree iff R_r is non-separable, so the descent trigger is exactly the non-separable residual locus. [CITED, GMN Lemma 3.11(3); NOVEL Lean encoding, ported from `OMCell.Child`.]

Definition 5.7 (OM cell: one Newton side $\times$ one residual stratum). [lean: `LeanUrat.OM.M5.OMCell`] An *OM cell* of T is

$$\text{OMCell } T := (\text{polygon} : \text{L4.LatticePolygon}, S :_r [Y], h_{S|} : S \mid R_r, d_S :, h_d : d_S = S.\text{natDegree}, \text{children} : \text{List}$$

one order- r Newton-polygon face (`polygon`, the M3 lattice face) carrying one residual-shape stratum S (a genuine factor of R_r , $h_{S|}$) of degree d_S , together with its descent children. This is the count shadow of `PadicMeasure.OMCell` with the `hStratumDvd` tie kept (cells are strata *of* the real residual, not free polynomials). The leaf/descent dichotomy:

- **leaf**: a multiplicity-1 irreducible residual factor S (so `children` = `[]` on that stratum) yields an irreducible p -adic factor with explicit $(e^*, f^*) = (e_1 \cdots e_r, f_0 \cdots f_r)$ (GMN Cor 1.20/3.8);
- **descend**: a multiplicity- ≥ 2 factor populates `children` with a `Child` of strictly smaller cluster size (`def:om-cell-child`).

[CITED, GMN Cor 3.8 dichotomy + Def 2.21 strata; NOVEL Lean encoding.]

Lemma 5.8 (Descent children carry repeated factors (projection)). [lean: `LeanUrat.OM.M5.descend'children'of factors`] For every T , every cell $c \in \text{cells } T$ and every $ch \in c.\text{children}$ there exist $P :_r [Y]$ and $\mu :$ with $2 \leq \mu$, $P^\mu \mid R_r$, and $cs(ch.\text{node}) = \mu$.

Proof sketch. Pure projection out of `Child`, exactly `PadicMeasure.descend_children_of_factors`: return $\langle ch.P, ch.\mu, ch.h_\mu, ch.h_l, ch.h_{cs} \rangle$. No axiom; the witness lives in the data. As in `PadicMeasure` this pin is faithfulness-only and *inert* on the counting path (M8 erases each child to `ch.node` and consumes only the size drop). [NOVEL, trivial.] $\square$

5.2 M6: well-founded termination by degree descent

Definition 5.9 (The strict descent measure: total degree). [lean: `LeanUrat.OM.M6.treeSize`] The descent measure of an `OMType` T is its *cluster size* $cs(T)$, identified with the count-native `treeSize` that the recursion descends on (cf. `MontesAxiom.MontesData.treeSize : ClusterShape $\rightarrow$ and PadicMeasure.OMShape.size). On the abstract ClusterShape index, treeSize is the total number of leaves of the decorated OM tree; on OMType it is the root cluster size. The well-founded relation is InvImage (<) treeSize on the shapes (Nat.lt_wfRel pulled back along treeSize by InvImage.wf), the exact input clusterCount/clusterMeasureModel recurse on via termination.by treeSize T / decreasing.by. [SUBSTITUTION, see rem:substitution.]`

Lemma 5.10 (Each descent child strictly decreases the measure). [lean: LeanUrat.OM.M6.child'size'lt]
 For every T , every cell $c \in \text{cells } T$ and every child $ch \in c.\text{children}$, $cs(ch.\text{node}) < cs(T)$.

Proof sketch. The child's size is the factor multiplicity, $cs(ch.\text{node}) = ch.\mu$ (def:om-cell-child, h_{cs}). We bound $ch.\mu$ by the parent measure. Two ingredients:

1. *Factor multiplicity $\leq$ residual degree.* From $h_1 : P^\mu \mid R_r$ and $R_r \neq 0$ (the leading coefficient of a residual is nonzero, GMN Def 2.21; in Lean $R_r.\text{Monic}$ from M4 `def:residual-poly`), `natDegree_le_of_dvd` gives $\mu \cdot \deg P = \deg(P^\mu) \leq \deg R_r = d_r$, and $\deg P \geq 1$ (an irreducible residual factor is non-constant), so $\mu \leq d_r$. More precisely, summing over the distinct irreducible factors $P_j^{\mu_j} \mid R_r$ via `Polynomial.natDegree_prod` gives $\sum_j \mu_j \deg P_j = d_r$, whence each $\mu_j \leq d_r$ and (for a genuine *descent* factor, $\mu_j \geq 2$, $\deg P_j \geq 1$) the strict slack $\mu_j < d_r$ unless $R_r = P_j^{\mu_j}$ with $\deg P_j = 1$ — the pure self-loop case excluded below.
2. *Residual degree $\leq$ parent cluster size.* The order- r residual has degree $d_r = \ell(S)/e \leq cs(T)$ (M4 degree formula `lem:residual-degree`: the residual degree is the side length over the ramification, which is at most the cluster size, the multiplicity the parent itself was formed at). Hence $cs(ch.\text{node}) = \mu \leq d_r \leq cs(T)$, and the descent inequality is strict because a multiplicity- ≥ 2 *proper* factor leaves residual degree $> \mu$ for at least one cofactor, equivalently the descent never reproduces the full cluster: $cs(ch.\text{node}) < cs(T)$.

This is precisely the data carried by `PadicMeasure.descend_size_lt` (there an axiom on `OMShape`); here it is *proved* from the M4 degree formula plus `natDegree_le_of_dvd/natDegree_prod`, with the self-loop (cs not dropping) isolated as the unique non-progressing cell that M8's `countPivot` resums — not a child. **[SUBSTITUTION for the bound source; the inequality itself is NOVEL/PROVED here, replacing the `descend_size_lt` axiom.]** $\square$

Remark 5.11 (Why degree descent, not GMN Thm 4.18). GMN's full termination (Thm 4.18, the Theorem of the Index, $\text{ind}(f) \geq \sum_r \text{ind}_r(f)$ with equality iff $\text{ind}_{r+1} = 0$; Cor 4.19) gives a sharp index bound on the descent depth. Formalising the index theorem requires the full higher-order valuation theory v_r (GMN Def 2.5), the Newton-polygon index identities, and defectlessness of K/p — a multi-month effort, **out of scope (DEFERRED)**. We *substitute* a strictly weaker but sufficient termination: the descent measure cs strictly decreases at every child (`lem:child-size-lt`), so the recursion is well-founded with depth $\leq cs(\text{root}) \leq n$. This suffices for every downstream consumer: M8's `clusterCount` is defined by `termination_by treeSize` and needs only the strict drop, and the depth bound `treeSize  $\leq n$` is exactly the `finiteTermination` obligation. Crucially, the bound “factor degrees sum to the residual degree $\leq$ cluster size $\leq n$ ” is purely combinatorial and **automatically p -independent** (it never mentions p or q), matching the GMN observation that the wild/inseparable case is generic, not exceptional. We lose only the *sharpness* of the depth bound, never its existence. **[SUBSTITUTION; the deferred sharp bound is GMN Thm 4.18.]**

Theorem 5.12 (`treeSize  $\leq n$` by strong induction on degree). [lean: LeanUrat.OM.M6.treeSize'le'degree]
 For an OM type T formed in the descent of a monic degree- n separable f , `treeSize  $T \leq n$` ; equivalently the number of leaves of the OM tree is at most n .

Proof sketch. By strong induction on the cluster size, in the `Nat.strongRecOn` shape used in `OMInduction.clusterVol.isRational` (which itself runs strong induction on `OMShape.size`). Reduce to: $\forall m, \forall T, cs(T) = m \rightarrow \text{treeSize } T \leq n$. The tree of T decomposes (M7, the cell partition) as the disjoint union over cells $c \in \text{cells } T$ of the leaves contributed by each residual stratum: a multiplicity-1 (leaf) stratum contributes one leaf carrying (e^*, f^*) with $\sum e^* f^*$

summing into n (GMN $\sum e_i f_i = [L : K]$, defectless); a multiplicity- ≥ 2 (descend) stratum contributes its children's leaves. By `lem:child-size-lt` each child has $cs(ch.node) < cs(T) = m$, so the IH applies, and the total leaf count over children is bounded by $\sum_{ch} \text{treeSize}(ch.node) \leq \sum_{ch} cs(ch.node)$. Since the factor multiplicities sum to at most $d_r \leq cs(T)$ and ultimately into the degree budget n (each leaf consumes ≥ 1 of the n slots $\sum e_i f_i = n$), the total is $\leq n$. Concretely the bound chains $\text{treeSize } T \leq \sum_{\text{leaves}} 1 \leq \sum_{\text{leaves}} e^* f^* = [L : K] \leq n$. The only Lean subtlety is the well-founded recursion obligation: the leaf-count function is defined by `termination_by cs(T)` with `decreasing_by` discharged by `lem:child-size-lt` (the same shape as `clusterCount`'s `decreasing_by` exact `hdsc T c.1 c.2 ch.1 ch.2`), the membership proofs threaded by `List.attach`. **[NOVEL/SUBSTITUTION: the degree-budget bound; depends on M7's partition for the exhaustive leaf decomposition.]** $\square$

5.3 Discharge of the MontesData obligations

Theorem 5.13 (Discharge of `cells_descend`). `[lean: LeanUrat.OM.M6.cells_descend]` With `cells` := the M7 cell list and `treeSize` of `def:descent-measure`, the field type

`cells_descend` : $\forall (T : \text{ClusterShape}), \forall c \in \text{cells } T, \forall ch \in c.\text{children}, \text{treeSize } ch < \text{treeSize } T$

of `MontesAxiom.MontesData` is inhabited.

Proof sketch. Direct restatement of `lem:child-size-lt` along the M9 decoding `ClusterShape` $\rightarrow$ `OMType` (M7 maps each `ClusterShape` to its `cells` : `List CountCell`, whose `children` : `List ClusterShape` are the decoded `ch.node`). The inequality `treeSize ch < treeSize T` is $cs(ch.node) < cs(T)$ transported through the decoding. This is exactly the hypothesis `hdsc` that `clusterCount` (M8) and `clusterCount_rec` consume in their `decreasing_by`. **[NOVEL; discharges the count shadow of the `PadicMeasure.descend_size_lt` axiom as a `MontesData` field.]** $\square$

Theorem 5.14 (Discharge of `finiteTermination`). `[lean: LeanUrat.OM.M6.finiteTermination]` The field type

`finiteTermination` : $\forall (\sigma : \text{FactorizationType}), \sigma.\text{degree} = n \rightarrow \forall T \in \text{shapesOf } \sigma, \text{treeSize } T \leq n$

of `MontesAxiom.MontesData` is inhabited.

Proof sketch. Apply `thm:treesize-le-n` to each $T \in \text{shapesOf } \sigma$. The hypothesis $\sigma.\text{degree} = n$ (i.e. $\sum_i e_i f_i = n$, `FactorizationType.degree`) supplies the degree budget the induction consumes: every shape of type σ arises in the descent of a monic degree- n f , so its leaf count is $\leq n$. This is GMN's qualitative termination (Cor 4.19: the descent of a separable degree- n f terminates) reduced to the degree-decrease substitution; the sharp index bound (Thm 4.18) is **not** needed and is deferred. Together with `thm:cells-descend-discharge`, the two “`ax : dichotomy`” `MontesData` fields (`cells_descend`, `finiteTermination`) are discharged from M4/M5/M6 with no new axiom. **[SUBSTITUTION for GMN Thm 4.18; PROVED via degree descent.]** $\square$

Remark 5.15 (What M5/M6 hand to M7/M8). M7 (`CellPartition`) consumes `def:om-cell` to build `cells` : `ClusterShape` $\rightarrow$ `List CountCell` and prove exhaustiveness (the strata reconstruct the residual factorization, $h_{S|}$ summing over factors R_r). M8 (`LevelNCount`) consumes `lem:child-size-lt/def:descent-measure` as the `hdsc` argument of `clusterCount`, and `thm:treesize-le-n` as the depth bound under which the level- N counting bijection's recursion terminates. The strict-descent and depth-bound facts proved here are p -independent and feed the well-founded recursion `OMInduction.clusterVolIsRational` shadow on the counting side (`clusterCount`). **[NOVEL bookkeeping.]**

6 Layer 3 — The level- N counting bijection and the real instance (M7–M9)

This layer is the payoff. M7 turns a real OM type into its list of counting cells and proves the strata *partition* the decided polynomials (discharging `MontesData.partition`). M8 is the long pole: it identifies the level- N stratum count with a product (box lattice volume) $\times$ (residual count) $\times$ ($\prod$ children), via a constant-fiber Hensel-lift coset argument, and passes to the density limit (discharging `boxHaarEquidist` and `nodeMeasure.boxSum`). M9 assembles the real `CountingModel/MontesData/DensityFoundation` and fires —`Goal.goalttheoremmontest, making it unconditional` `flagged elementary root – count bound`.

Throughout: $K/\mathbb{Q}_p$ finite, O_K its integers, $q = |k_K|$ the residue cardinality (a power of p), n the degree; $f = X^n + \sum a_i X^i \in O_K[X]$ monic; N the truncation level; the box is `monicBox(N, n)`, the monic degree- n polynomials over $O_K/\mathfrak{p}^N \cong \text{ZMod}(p^N)$? (via M1’s —`PadicInt.toZModPow`—), of cardinality $(p^N)^n$. We freely reuse the Layer-0/1/2 nodes by their labels: `lem:m0-residual-count` (M0 residual oracle, $= (q^\delta)^{d_S-1}$), `def:quotient-box`, `lem:card-monicbox` (M1 box and $\#\text{monicBox}(N, n) = (p^N)^n$), `def:omtype`, `def:om-cells-data`, `lem:om-leaf-descent` (M5 real OM type, its cell data, leaf/descent dichotomy), `lem:treesize-le-n` (M6 termination, $\text{treeSize} \leq n$), and the L4 cell-volume identity `lem:l4-cellvolume`.

6.1 M7 — CellPartition: real cells and the exhaustive partition

Definition 6.1 (Counting cells of a real OM type). [`lean: LeanUrat.OM.M7.cellsOfType`] Let T be a real order- r OM type (`def:omtype`) of an f at degree n . Its *counting cells* `cellsOfType(T) : List MontesAxiom.CountCell` are obtained as follows. For each finite-slope side S of the order- r Newton polygon N_r of T (`def:newtonpoly-side`), and each pairwise-distinct monic irreducible factor ψ of the residual polynomial $R_S \in F_r[Y]$ (`def:residual-poly`) with multiplicity a_ψ in the factorization $R_S \sim \prod_\psi \psi^{a_\psi}$ (GMN Thm 3.7), emit one —`CountCell`— with fields

$$\text{ds} := \deg \psi, \quad \delta := f_0 f_1 \cdots f_{r-1} \quad (= [F_r : F_q], \text{ so the residual count is over } F_{q^\delta}),$$

$$\text{polygon} := \text{the L4 lattice face of } S \text{ (def:side-to-l4face)}, \quad \text{children} := \begin{cases} [] & a_\psi = 1 \text{ (leaf, Cor 3.8),} \\ [T_\psi] & a_\psi \geq 2 \text{ (descend, Lemma 3.} \end{cases}$$

where T_ψ is the order- $(r+1)$ child OM type sprouted from the repeated factor ψ (the multiplicity- ≥ 2 descent of `lem:om-leaf-descent`; `children` is a singleton because the descent attaches the single key polynomial ϕ_{r+1} lifting ψ). The shape T corresponds to a —`ClusterShape`— via the M5 encoding `def:shape-of-type`, and we set `cellsD(shape(T)) := cellsOfType(T)`. **NOVEL** (the count shadow of —`PadicMeasure.omCells`—, now built from real M4/M5 residual data; cf. `MontesAxiom.CountCell`).

Lemma 6.2 (The descent children are strictly smaller). [`lean: LeanUrat.OM.M7.cells'descend`] With `treeSizeD := treeSize` from `lem:treesize-le-n` (M6), for every shape T , every cell $c \in \text{cells}_D(T)$ and every child $\text{ch} \in c.\text{children}$, `treeSizeD(ch) < treeSizeD(T)`. **NOVEL** (discharges the field —`MontesData.cellsddescend`—).

Proof sketch. A child is emitted only for a multiplicity- ≥ 2 residual factor ψ , whose child type T_ψ has cluster size `cs(Tψ) = aψ` and total degree bounded by $\deg \psi \cdot a_\psi \leq \deg R_S \leq \deg_S$, strictly below the parent’s total degree by the M6 strict-descent measure (`lem:treesize-le-n`, the degree-decrease well-foundedness). Concretely this is exactly the M6 lemma —`treeSizecchildt|restated on def:cells-of-ty`

Lemma 6.3 (Exhaustive residual partition / the strata partition the decided box). [lean: LeanUrat.OM.M7.partition] Fix a degree- n type σ and a level N . Let $\text{shapesOf}(\sigma)$ be the finite set of real OM shapes T with $\text{type}(T) = \sigma$ (`def:shape-of-type`; finite by `lem:treesize-le-n`). Then the level- N strata partition the decided polynomials:

$$M.\text{decidedCount } \sigma \ N = \sum_{T \in \text{shapesOf}(\sigma)} \text{stratumCount } T \ N,$$

where $\text{stratumCount } T \ N := (\#\{f \in \text{monicBox}(N, n) \text{ decided to shape } T\}) \in \mathbb{Q}$ (`def:stratumCount`). **NOVEL** (discharges —MontesData.partition—).

Proof sketch. A coset $f \in \text{monicBox}(N, n)$ is *decided to be type* σ iff its OM tree, computed from the digits below $\mathfrak{p}^N$, terminates with leaf data assembling to σ (GMN Cor 4.19; this is the N -truncated OM algorithm). The OM tree of a fixed lift is *unique* (the dissections of GMN Thm 3.1/3.7 are by pairwise-distinct slopes and pairwise-distinct residual factors, hence disjoint), so the map “decided $f \mapsto$ its OM shape T ” is well-defined and lands in $\text{shapesOf}(\sigma)$. Exhaustiveness and disjointness reduce to the residual *factorization* being a genuine partition of the residual roots: at each side S , $R_S = \prod_{\psi} \psi^{a_{\psi}}$ with the ψ pairwise coprime (`lem:residual-factorization`, i.e. —Polynomial.prod— of the distinct irreducible factors with multiplicity equals R_S), so the strata indexed by (S, ψ) are disjoint and cover all decided cosets. Formally:

1. Build the finite partition of the decided set indexed by $T \in \text{shapesOf}(\sigma)$ via —Finset.sum fiberwise|over the (decided) classifiers $\text{shapeOf}:\{\text{decided } f\} \rightarrow \text{ClusterShape}$.
2. Identify each fiber’s cardinality with $\text{stratumCount } T \ N$ (`def:stratumCount`, by definition the count of f with $\text{shapeOf}(f) = T$).
3. Cast the cardinalities to $\mathbb{Q}$ (—Nat.cast_{s,um}|); the equation is then $|Finset.card_{e,q,um,c} \text{ fiberwise}| \text{ transported to } \mathbb{Q}$. The only nontrivial input is that the residual factorization at each side is the full coprime factorization (`lem:residual-factorization`, M4); the rest is finite combinatorics over the box. $\square$

6.2 M8 — LevelNCount: the counting bijection (the long pole)

Definition 6.4 (The level- N stratum count). [lean: LeanUrat.OM.M8.stratumCount] For a real OM shape T at degree n and level N ,

$$\text{stratumCount } T \ N := (\text{Nat.card } \{f \in \text{monicBox}(N, n) : \text{shapeOf}(f) = T\}) : \mathbb{Q},$$

the number of monic degree- n cosets mod $\mathfrak{p}^N$ whose N -truncated OM tree has shape T . This is the count realizing the M1 field —MontesData.stratumCount—. **NOVEL**.

The crux: the constant-fiber Hensel-lift lemma. The whole bijection rests on understanding how the box count changes when we refine the truncation level $N \rightarrow N + 1$. The reduction map

$$\pi_N : \text{ZMod}(p^{N+1}) \twoheadrightarrow \text{ZMod}(p^N)$$

(M1’s —ZMod.castHom— / the projective system underlying —PadicInt.toZModPow—) has all fibers of cardinality p (a single residue digit). Lifting this coordinatewise to monic degree- n polynomials, the box-refinement map $\Pi_N : \text{monicBox}(N + 1, n) \twoheadrightarrow \text{monicBox}(N, n)$ has every fiber of size p^n (one new digit per coefficient). The content of M8 is that, *restricted to one fixed cell* $c = (S, \psi)$, the number of those p^n lifts that *stay in the cell* is a *constant* not depending on which decided f we lift — and that constant is exactly the per-coordinate box/shell ratio times the residual-configuration count.

Lemma 6.5 (Constant-fiber lemma over a fixed cell). [lean: LeanUrat.OM.M8.constantFiber] Fix a real shape T , a cell $c \in \text{cellsOfType}(T)$, and a level $N \geq N_0(c)$ (above the finite level N_0 at which the cell's slope side and residual stratum are already certified — the Newton-polygon vertices and the residual leading/trailing coefficients have stabilized mod $\mathfrak{p}^{N_0}$, GMN Thm 2.11/Def 2.21). Then for *every* coset $f \in \text{monicBox}(N, n)$ already decided to lie in the stratum of c , the number of lifts $\tilde{f} \in \Pi_N^{-1}(f) \subset \text{monicBox}(N+1, n)$ that remain in the stratum of c is a constant

$$\kappa(c) := \underbrace{(q^\delta)^{d_S-1}/(q^\delta)^{d_S}}_{\text{residual ratio (M0)}} \cdot \underbrace{\prod_i \rho_i(c)}_{\text{column box/shell ratios (L4)}} \cdot p^n,$$

in the sense that $\#\Pi_N^{-1}(f) \cap (\text{stratum } c) = \kappa(c) \cdot p^{-?}$ is *independent of f* . Equivalently, the normalized per-step ratio

$$\frac{\text{stratumCount}_c T(N+1)}{\text{stratumCount}_c T N}$$

is a constant $r(c)$ depending only on c , not on N (for $N \geq N_0(c)$), where stratumCount_c is the contribution of cell c alone. **NOVEL** (the genuine Hensel-lift content; most of the Lean effort lives here).

Proof sketch. Work coordinatewise. A monic degree- n f is its coefficient vector $(a_0, \dots, a_{n-1}) \in (O_K/\mathfrak{p}^N)^n$, and Π_N is the product reduction. The cell condition factors as a *conjunction of independent coordinate conditions* plus a residual condition:

1. **(Newton box, per coordinate).** The side S of the order- r Newton polygon imposes, on each abscissa i , a valuation condition $v_i(f) \geq \lceil h_i \rceil$ (non-vertex column, a *ball*) or $v_i(f) = \lceil h_i \rceil$ (vertex column, a *shell*). Refining $N \rightarrow N+1$ multiplies the count of admissible digits at column i by p uniformly when the condition is a ball (the new digit is free) and by p with the leading-digit shell already fixed — in both cases by a factor independent of f . The product of these per-column ratios is exactly the L4 per-column box/shell measure `lem:l4-cellvolume` read *over the finite ring $\text{ZMod}(p^N)$ rather than via Haar*: this is the load-bearing re-proof `lem:l4-coordinate-residue-indep` below.
2. **(Residual configuration, per stratum).** On the side S the residual polynomial $R_S(f) \in F_r[Y]$ is determined by the digits of f at the abscissae of S modulo one further level; the condition “ $R_S(f)$ lies in the ψ -stratum (i.e. $\psi^{a_\psi} \mid R_S$, the others coprime)” is a condition on a fixed finite-field residual that, by GMN Def 2.21, depends only on the bottom digit of those coefficients. Refining $N \rightarrow N+1$ leaves the residual configuration count at the constant M0 value $(q^\delta)^{d_S-1}$ (`lem:m0-residual-count`); the extra digit is again free.
3. **(Independence).** The Newton-box conditions (on the *valuations* of the coefficients) and the residual condition (on the *leading units* of the coefficients on the side) act on disjoint digit slots, so by the Chinese-remainder/coordinate splitting of $\text{ZMod}(p^{N+1}) \cong (\text{digits})$, the joint count is the product. This is `lem:l4-coordinate-residue-indep`.

Putting (1)–(3) together, $\Pi_N^{-1}(f) \cap (\text{stratum } c)$ has the *same* cardinality for every in-stratum f , equal to (product of per-column digit ratios) $\times$ (residual digit freedom), all f -independent. The mathlib tools: —`ZMod.castHom_surjective` | *and the fibercardinality* — $|\ker(\pi_N^{N+1})| = p$ via —`ZMod.card`— and —`Nat.card_eq` | *of the kernel of the level map (the N -vs- $(N+1)$ digit)*, $|P_{\text{adicInt}}.\ker \iota \circ \text{ZModPow}| / |P_{\text{adicInt}}.\text{appr} \text{ZMod}(p^?)|$. $\square$

Lemma 6.6 (Coordinate-residue independence over $\text{ZMod}(p^N)$ — the L4 `hfactor` re-proof). [lean: LeanUrat.OM.M8.cellVolume`factor`zmod] For a fixed cell c with L4 polygon face Π and residue size $Q = q^\delta$, the finite count of in-cell coordinate vectors over $\text{ZMod}(p^N)$ factors as a product over the polygon’s columns:

$$\frac{\#\{(a_i) \in (\text{ZMod}(p^N))^? : \text{cell condition of } \Pi\}}{(p^N)^?} = \prod_{i:\text{Fin } \Pi.\text{width}} \text{columnMeasure } \Pi \, Q \, i.$$

Hence the abstract —hfactor— hypothesis consumed by `lem:14-cellvolume` (—L4.cellVolume_eq|) is, over the finite a proved counting identity, not a measure assumption: the per-coordinate conditions are independent because the coefficients a_i occupy disjoint digit slots (—Fintype.card_pi|/|Finset.prod|over|Fin|factoring of the bearing lemma that turns L4’s cited |hfactor|product-over-columns into a finite count; it is what discharges the analysis).

Proof sketch. The cell condition is a conjunction $\bigwedge_i C_i(a_i)$ with C_i a per-coordinate valuation/shell condition (`def:side-to-14face`). Over $\text{ZMod}(p^N)$ the coefficient vector lives in $\prod_i \text{ZMod}(p^N)$ (a finite product), so by —Fintype.card_pi|the count of vectorssatisfying $\bigwedge_i C_i$ is $\prod_i \#\{a_i : C_i(a_i)\}$. Dividing by the total $(p^N)^? = \prod_i p^N$ gives the product of normalized per-column counts; each normalized per-column count equals `columnMeasure` $\Pi \, Q \, i$ by the elementary ball/shell count over $\text{ZMod}(p^N)$ (a ball $v(a_i) \geq h$ has p^{N-h} residues, ratio Q^{-h} ; a shell $v(a_i) = h$ has $p^{N-h} - p^{N-h-1}$, ratio $(1 - Q^{-1})Q^{-h}$ — matching —L4.columnMeasure—’s two branches). Then `lem:14-cellvolume` collapses the product to `bb1Value` $\Pi \, Q$. $\square$

Theorem 6.7 (The level- N counting bijection). [lean: LeanUrat.OM.M8.stratumCount`factor] For every real shape T at degree n and every level N large enough that all cells of T are certified ($N \geq \max_c N_0(c)$),

$$\text{stratumCount } T \, N = \sum_{c \in \text{cellsOfType}(T)} \underbrace{\left(\text{bb1Value } c.\text{polygon } (p^N) \text{ box term} \right)}_{\text{L4}} \cdot \underbrace{\left((q^\delta)^{d_S-1} \right)}_{\text{M0 residual}} \cdot \prod_{ch \in c.\text{children}} \text{stratumCount } T \, N$$

up to the uniform box normalization $(p^N)^n$ (made precise in `thm:stratum-limit`). The “ $\prod$ children” is the recursive count of the descent sub-strata, the well-founded recursion guaranteed by `lem:cells-descend`. **NOVEL.**

Proof sketch. Decompose `stratumCount` $T \, N$ over the cell partition (`lem:m7-partition` localized to shape T): the in-shape cosets split disjointly across the cells $c = (S, \psi)$ (distinct slopes, distinct residual factors), so `stratumCount` $T \, N = \sum_c \text{stratumCount}_c \, T \, N$. For each cell the per-cell count is, by `lem:constant-fiber` run inductively from the certification level $N_0(c)$ up to N , the product of: the L4 Newton-box count over $\text{ZMod}(p^N)$ (`lem:14-coordinate-residue-indep`, collapsing to `bb1Value`); the M0 residual-configuration count $(q^\delta)^{d_S-1}$ (`lem:m0-residual-count`, the squarefree complement —residualBoxCount— read as the stratum count); and the recursive descent count over $c.\text{children}$. The recursion is well-founded by `lem:cells-descend` (children strictly smaller —tree-Size—), so the product-over-children is a legitimate —termination_{by}|recursionmirroring|clusterCount|.TheLean

Theorem 6.8 (The density limit: `nodeMeasure.boxSum` and `boxHaarEquidist`). [lean: LeanUrat.OM.M8.stratum`tendsto] Define the per-box measure `boxMeasure` $c \, q := \text{countCellCoeff } c \, q = (q^\delta)^{d_S-1} \cdot \text{bb1Value } c.\text{polygon } q$ (M1 —MontesAxiom.countCellCoeff—). Then:

1. (`boxHaarEquidist`) `boxMeasure` $c \, q' = \text{countCellCoeff } c \, q'$ for all $q' > 1$, by definition — the per-box Haar volume IS the proved box-volume $\times$ residual-count (`lem:14-cellvolume` for the box factor over ZMod , `lem:m0-residual-count` for the residual factor). This is now a `rfl`/proved identity, not an axiom.

2. (`nodeMeasure_boxSum`) the normalized stratum measure converges:

$$\lim_{N \rightarrow \infty} \frac{\text{stratumCount } T \ N}{(p^N)^n} = \frac{\sum_{c \in \text{cells}(T)} \text{boxMeasure } c \ q \cdot \prod_{\text{ch} \in c.\text{children}} \text{clusterCount } \dots \text{ch } q}{\text{countPivot } (\text{treeSize } T) \ q}.$$

NOVEL (discharges the two minimal-axiom fields —`boxHaarEquidist`— and —`nodeMeasure_boxSum`— of *MontesData*).

Proof sketch. For (1): unfold `boxMeasure` and —`countCellCoeff`—; the box factor equals `bb1Value` over $\mathbb{Z}\text{Mod}(p^N)$ in the limit by `lem:14-coordinate-residue-indep` + `lem:14-cellvolume`, and the residual factor is the M0 count `lem:m0-residual-count`; both are the literal factors of —`countCellCoeff`—, so the field is `rfl` once `boxMeasure` is *defined* as `countCellCoeff` (the witness template —`Witness.trivD`— sets it so; the real instance does too).

For (2): from `thm:levelN-bijection`, divide by $(p^N)^n$. The L4 box term `bb1Value c.polygon (p^N)` over $\mathbb{Z}\text{Mod}(p^N)$ tends to the q -value `bb1Value c.polygon q` as $N \rightarrow \infty$ (the per-level box ratio stabilizes by `lem:constant-fiber`: the per-step ratio is the constant $r(c)$, and the geometric tail $\sum_N r(c)^N$ over the unique self-loop resums to the closed pivot $1/(1-q^{-w})$, $w = \text{selfLoopExponent}$ — exactly —`countPivot`— via —`L5fix.selfLoop_geometric`—/—`NestedCollapse.geometricLimit_of_selfLoop`—). *The residual-foundedness, recursing the same limit on strictly smaller $|treeSize|$. Combining, then the normalized stratum measure converges by `Filter.Tendsto.div`, `|tendsto_finsetSum|`, `|tendsto_const_nhds|` for the N -constant residual, the geometric-series limit `hasSum_geometric_fit_one` repackaged through `L5fix.selfLoop_geometric`, and the recursion—`unfold MontesAxiom.clusterCount_rec`— to match the RHS shape verbatim. $\square$*

Remark 6.9 (Why this discharges the minimal axiom). The two fields —`boxHaarEquidist`— and —`nodeMeasure_boxSum`— are the only measure content carried by *MontesData* (*MontesAxiom* § “minimal irreducible wise axiom”). M8 makes both theorems over the counting model: *there is no Haar measure invoked* | the “box measure” is (M1), and “equidistribution” is the M0 finite-field count. The downstream —*MontesData.nodeMultiplicativity*— (already a theorem in *MontesAxiom*, derived from these two fields via —`clusterCount_boxSum`—) therefore becomes surjective.

6.3 M9 — RealInstance: assemble and fire the goal

Definition 6.10 (The real counting model). [`lean: LeanUrat.OM.M9.realM`] For a fixed $K/\mathbb{Q}_p$ with residue cardinality $q = p^{f_K}$ and degree n , define `realM : CountingModel q n` with (cf. the shape template —`Witness.trivM`—):

- `hq := 2 ≤ q` (residue cardinality of a nontrivial residue field);
- `decidedCount σ N := # {f ∈ monicBox(N, n) decided to type σ},` cast to $\mathbb{Q}$;
- `undecidedCount N := # {f ∈ monicBox(N, n) not yet decided};`
- `typeMenu :=` the finite set of degree- n types realized (finite by `lem:treesize-le-n` / GMN Cor 4.19);
- `box_partition:` `decided + undecided = (p^N)^n = #monicBox(N, n)` (`lem:card-monicbox`; the box is exhaustively classified into decided types $\cup$ undecided);
- `decided_mono:` refining a decided coset keeps it decided (monotone), since the OM decision at level N is preserved under reduction π_N (`lem:constant-fiber` is the refinement map; decided strata only grow);
- `densityVal σ := ∑_{T ∈ shapesOf(σ)} coeff_D T q` (the genuine OM tree-sum, = the limit by `thm:stratum-limit`);

- `decided_tendsto`: from `thm:stratum-limit` summed over `shapesOf( $\sigma$ )` via `—Decomposition.decidedMeasure`
`from lem:constant-fibermonotonicity|masscertifiedbetweenlevelsNandKwasundecidedatN, so the incren`

NOVEL (the long-missing real model).

Definition 6.11 (The real `MontesData`). [`lean: LeanUrat.OM.M9.realD`] Define `realD : MontesData  $q$  n realM` (cf. `—Witness.trivD—`), all five GMN-shadow fields now *proved* by Layer-3 nodes:

- `shapesOf`, `treeSize := M5/M6` (`def:shape-of-type`, `lem:treesize-le-n`);
- `cells := def:cells-of-type`; `cells_descend := lem:cells-descend`;
- `stratumCount := def:stratumCount`; `partition := lem:m7-partition`;
- `boxMeasure  $cq$  := countCellCoeff  $cq$` ; `boxHaarEquidist := thm:stratum-limit(1) (rfl)`;
`nodeMeasure_boxSum := thm:stratum-limit(2)`;
- `finiteTermination := lem:treesize-le-n` (`M6`, `treeSize  $T \leq n$`);
- the undecided-tail fields (`separableUndecidedMeasure`, `discriminantTailMeasure`, `undecided_le_split`,
`sepTail_tendsto`, `discriminantTail_envelope`) := `lem:sep-tail` and `lem:disc-tail`.

NOVEL. Feeding this into the already-proved `—MontesData.undecidedVanishes—` and `—MontesData.countingDe`

Lemma 6.12 (Separable tail vanishes). [`lean: LeanUrat.OM.M9.sepTail_tendsto`] `separableUndecidedMeasure  $N \rightarrow 0$` . **SUBSTITUTION/CITED** (GMN Thm 4.18 / Cor 4.19, `lem:treesize-le-n`).

Proof sketch. Every separable degree- n f resolves its OM tree at finite OM order (`lem:treesize-le-n`: the degree-decrease descent terminates, `treeSize  $\leq n$`), hence is decided at some finite precision N_f . The per-level separable-undecided pool is monotone decreasing as each f drops out at its own N_f ; the pointwise drain gives $\rightarrow 0$ (`—tendstoatTopofmonotone|/amonotone—boundedargumenton the finitemenu`, since

Lemma 6.13 (Discriminant-tail decay — the DEFERRED elementary root-count). [`lean: LeanUrat.OM.M9.discriminantTail_envelope`] There is a constant $C \geq 0$ with `discriminantTailMeasure  $N \leq C \cdot q^{-N}$` , i.e. the non-separable locus $\{f : \Delta(f) \equiv 0 \pmod{\mathfrak{p}^N}\}$ has counting density $\leq C \cdot q^{-N}$, where Δ is the (fixed, nonzero) discriminant polynomial of the generic degree- n f . **DEFERRED** (the one elementary number-theory fact absent from `mathlib v4.31.0`: a nonzero polynomial’s mod- $\mathfrak{p}^N$ root-set over $\mathbb{Z}\text{Mod}(p^N)$ has density $\leq C \cdot q^{-N}$). It is carried as the explicit field `—discriminantTaileenvelope|(with constant|d`

Proof sketch. $\Delta \in O_K[X_0, \dots, X_{n-1}]$ is a fixed nonzero polynomial in the coefficients of f . Over a DVR, the density of the zero locus mod $\mathfrak{p}^N$ of a nonzero polynomial decays geometrically: each “new” zero must agree with $\Delta \equiv 0$ to one further digit, contributing a factor $\leq q^{-1}$ per level, giving the envelope $C \cdot q^{-N}$ with C a Hensel/Newton constant (degree-bounded). The $\rightarrow 0$ limit is then *proved* from this envelope by `—tendstopowatTopnhdszeroofitone|(alreadydoneinside|MontesData.discriminantT`
`countboundoverZMod( $\mathfrak{p}^N$ )` — is deferred; everything downstream of it is proved. $\square$

Definition 6.14 (The real density foundation). [`lean: LeanUrat.OM.M9.realF`] Define `realF : DensityFoundation` (cf. `—Witness.trivF—`) with `density  $n \sigma q := \sum_{T \in \text{shapesOf}(\sigma)} \text{coeff}_{\text{realD}} T q$` (the genuine OM tree-sum, = `realM.countingDensity  $\sigma$` by `thm:stratum-limit/—countingDensityeqsumcoeff|`), `clusterVolume  $T q := \text{coeff}_{\text{realD}} T q$` , `shapes := shapesOf`, `multiplicity := 1`, `decomposition` by `rfl`, and `density_nonneg` from `stratumCount` nonnegativity (counts are ≥ 0). **NOVEL**.

Theorem 6.15 (`goal_theorem_montes` fires unconditionally modulo `tame_functionalEquation`).
`[lean: LeanUrat.OM.M9.montes' unconditional]` For the real $(\text{realM}, \text{realD}, \text{realF})$ and any degree- n type σ ,
`—Goal.goaltheoremmontesrealMrealDσ realF (h) (hbridge)—` yields $\exists \text{num}, \text{den}$ with $\text{den} \neq 0$,
uniform rationality of the genuine counting density $q \mapsto \sum_T \text{coeff } T q$ at every $q > 1$, the tie
`realM.countingDensity σ = num(q)/den(q)` at this model's q , and `—IsPalindromic num den—`. The
bridge hypothesis `hbridge :  $\forall q' > 1, \text{realF.density } n \sigma q' = \sum_T \text{coeff } T q'$` holds by `rfl` (`def:real-foundation`
sets `density` to that sum). The result is unconditional modulo only `—tameffunctionalEquation|(the palindromy ha`
`tame, untouched) and the DEFERRED lem:disc-tail envelope.NOVEL(the integration; this is the deliverable' se`

Proof sketch. Identical wiring to `—Witness.montesffullinstance|, but with the real bundle :`

Rationality flows through `—MontesData.countingDensityisRational|(Decomposition), which depends only on Lean`
`limittie countingDensity σ =  $\sum_T \text{coeff } T q$  is —countingDensityeqsumcoeff|(Decomposition), now unconditional be`
`—MontesData.undecided Vanishes—` is unconditional: `sepTail_tendsto (lem:sep-tail)` and `discriminantTail_`
(from the `lem:disc-tail` envelope) both $\rightarrow 0$, squeezing undecided $\rightarrow 0$.

`hbridge` by `rfl`; then `—goaltheoremmontes|applies verbatim, its L7 identity—theorem transfer carrying|tameffunctionalEquation`
The axiom footprint of `thm:m9-goal` is exactly `{propext, Classical.choice, Quot.sound, tame_functionalEquation}`
plus the explicit hypothesis `lem:disc-tail` (carried as a `—MontesData—` field value, not an ax-
iom) — i.e. the conditional hypotheses currently carried in `[lean: “]urat` are discharged, and the real
instance is constructed. □

7 Closing the DAG (v2): the six glue nodes

The v1 reconciler found every CITED `lean_urat/mathlib` signature accurate at the cited lines, but the DAG was not formally closed: six glue nodes were missing and a layer of label drift and notation slips remained. This section writes the six nodes in full and records the canonicalization, after which every edge resolves and each of the five `MontesData` discharge targets is backed by a node with no unbacked step. The six nodes are, by gap: **GAP 1** `lem:residual-factorization` (M4, the complete coprime residual factorization); **GAP 4** `def:side-to-l4face` (M3→M7, the per-side L4 face) with its spec `lem:side-to-l4face-spec`; **GAP 2** `def:shape-of-type`, `def:shape-class`, `def:decode`, `lem:shape-transport` (M5/M7/M9, the `ClusterShape` ↔ `OMType` transport); **GAP 3** `lem:om-tree-uniqueness` (M5, the CITED GMN Thm 3.1/3.7 dissection/uniqueness import); **GAP 5** `lem:certification-level` (M8, the finite certification level $N_0(c)$); and **GAP 6** `lem:selfloop-ratio` (M8, the self-loop ratio resummation to `countPivot`). GAP 5 and GAP 6 are stated here in their canonical form for completeness and are physically inserted in the M8 body next to their consumers (`lem:constant-fiber`, `thm:stratum-limit`); GAP 1–4 are stated here and cross-referenced from M3/M4/M5/M7.

7.1 GAP 1 (M4): the complete coprime residual factorization

Lemma 7.1 (Complete coprime factorization of the residual polynomial). [`lean: LeanUrat.OM.M4.residualPoly` factorization] Let $k_S = F_r$ be the finite (hence perfect) residue field of the side S , and let $R := R_S(f) \in k_S[Y]$ be the residual polynomial (`def:residual-poly`), of degree $d = \ell(S)/e_S$ with nonzero constant and leading coefficients (`lem:residual-degree`, `lem:residual-endpoints`). Then there is a *finite, pairwise-distinct* family of monic irreducible polynomials $\psi_1, \dots, \psi_k \in k_S[Y]$ and exponents $a_{\psi_1}, \dots, a_{\psi_k} \geq 1$ such that

$$R = c \cdot \prod_{j=1}^k \psi_j^{a_{\psi_j}}, \quad c := R.\text{leadingCoeff} \in k_S^\times,$$

with the following structural data:

1. (**distinctness / pairwise coprimality**) the ψ_j are pairwise non-associated monic irreducibles, hence pairwise coprime: for $i \neq j$, `IsCoprime` $\psi_i \psi_j$ (equivalently $\gcd(\psi_i, \psi_j) = 1$ in the Euclidean domain $k_S[Y]$);
2. (**degree budget**) $\sum_{j=1}^k a_{\psi_j} \cdot \deg \psi_j = d = \deg R$;
3. (**multiplicity = cluster size at a descent factor**) for each j , the exponent $a_{\psi_j} = \text{Multiset.count } \psi_j \text{ (normalizedFactors } R)$ is the exact multiplicity of ψ_j in R , and this is precisely the cluster size of the child the stratum ψ_j spawns: $a_{\psi_j} = \text{cs}(\text{child}_{\psi_j})$ (`def:cluster-size`); the descent factors are exactly those ψ_j with $a_{\psi_j} \geq 2$, the leaf factors those with $a_{\psi_j} = 1$.

The factorization is **unique** up to reordering and unit (i.e. up to associates of the ψ_j), and exhaustive: *every* monic irreducible divisor of R occurs among the ψ_j .

Flag: NOVEL (the full coprime factorization as extractable data; GMN Def 2.21 / the factorization ψ -data behind the order-($r+1$) types of Thm 3.7). The underlying UFD/field factorization is CITED mathlib.

Proof sketch. Work in $k_S[Y]$, which is a PID/EuclideanDomain (it is a polynomial ring over a field, `Polynomial.instEuclideanDomain`), hence a UniqueFactorizationMonoid (`EuclideanDomain.instUniqueFact` or directly `Polynomial.instUniqueFactorizationMonoid` over a field). $R \neq 0$ since its leading coefficient $c = c_d \neq 0$ (`lem:residual-endpoints`), so its factorization multiset `normalizedFactors R` is well-defined and finite (`UniqueFactorizationMonoid.normalizedFactors`). The normalized prime associates over a field are exactly the *monic* irreducibles (the normalization unit of a polynomial over a field is its leading coefficient, `Polynomial.normalize_eq / Polynomial.Monic.normalize_eq_self`; `normalizedFactors_prod` gives $\prod(\text{normalizedFactors } R) = \text{normalize } R$, with the discarded unit $= c$). Concretely:

1. Let $\{\psi_1, \dots, \psi_k\} := (\text{normalizedFactors } R).toFinset$ be the *distinct* primes, monic by the normalization-over-a-field fact, and set $a_{\psi_j} := \text{Multiset.count } \psi_j (\text{normalizedFactors } R) \geq 1$. Then `Multiset.toFinset_prod_pow_count` (collecting a multiset product by its distinct entries) rewrites $\prod(\text{normalizedFactors } R) = \prod_j \psi_j^{a_{\psi_j}}$, and `normalizedFactors_prod` folds in the unit $c = R.\text{leadingCoeff}$ via `associated_iff_eq/normalize_eq` to give $R = c \cdot \prod_j \psi_j^{a_{\psi_j}}$.
2. *Pairwise coprimality.* Distinct monic irreducibles over a field are non-associated, and two distinct irreducibles in a GCDMonoid are coprime (`Irreducible + not associated  $\Rightarrow$  EuclideanDomain.isCoprime` or `Polynomial.isCoprime_of_isUnit_gcd`); membership in a Finset of distinct normalized factors gives the non-association, so `Multiset.Nodup` of `(normalizedFactors R).toFinset.val` discharges $i \neq j \Rightarrow \text{coprime}$.
3. *Degree budget.* Take $\deg$ of $R = c \cdot \prod \psi_j^{a_{\psi_j}}$: $\deg(c \cdot P) = \deg P$ for the unit c (`Polynomial.natDegree_C.mul` with $c \neq 0$), and $\deg \prod \psi_j^{a_{\psi_j}} = \sum_j a_{\psi_j} \deg \psi_j$ (`Polynomial.natDegree_prod` over the field, no zero factors; `Polynomial.natDegree_pow`). Equate to $d = \deg R$ from `lem:residual-degree`.
4. *Multiplicity = cluster size.* The exponent a_{ψ_j} is the `Multiset.count` multiplicity, exactly the μ of `def:cluster-size` (`cs(child) =  $\mu$  = Multiset.count  $\psi$  (factors  $R$ )`); for $a_{\psi_j} \geq 2$ this reproduces the descent witness (ψ_j, a_{ψ_j}) of `lem:descent-repeated-factor`, and $\neg \text{Squarefree } R$ holds iff some $a_{\psi_j} \geq 2$ (`UniqueFactorizationMonoid.squarefree_iff_nodup_normalizedFactors`: $R \text{ squarefree} \iff \text{the factor multiset is Nodup} \iff \text{all } a_{\psi_j} = 1$), which recovers `thm:residual-sep-sqfree` as the special case “ k distinct factors all of multiplicity 1.”

Hard point. The only genuinely new bookkeeping over `lem:descent-repeated-factor` (which only produced *one* repeated factor) is assembling the *full* multiset as the cell-indexing Finset and proving the product/degree/coprimality clauses simultaneously; the cleanest Lean shape is to expose the factorization as a single structure `ResidualFactorization R` carrying the Finset of distinct monic irreducibles, the count function $a_{(-)}$, and proofs of (1)–(3), so that `def:cells-of-type` consumes it directly (one cell per element of the Finset, with $d_S := \deg \psi_j$ and child cluster size a_{ψ_j}). Over a field there are no normalization subtleties beyond the leading-coefficient unit c ; perfectness of k_S (`thm:residual-sep-sqfree`) is *not* needed for the factorization itself (which holds over any field), only to identify “non-separable” with “some $a_{\psi_j} \geq 2$.” $\square$

What this node delivers downstream.

- `def:cells-of-type` (M7) emits *one* `CountCell` per distinct irreducible factor ψ_j of each side’s residual: the cell’s $d_S := \deg \psi_j$ (residual-stratum degree, fed to the $T_{BB3} = (q^\delta)^{d_S-1}$ count of `lem:nonsqfree-count`), and its `children: List ClusterShape` is the (possibly empty for $a_{\psi_j} = 1$) list of descent children. The pairwise coprimality (clause 1) is what makes the cells *disjoint* strata; the degree budget (clause 2) is what makes them *exhaustive* ($\sum_j a_{\psi_j} \deg \psi_j = \deg R$, no residual mass unaccounted for).

- **lem:m7-partition** (M7) uses clauses (1)–(2) as the residual-side half of the “strata partition the decided box” claim; **lem:om-tree-uniqueness** supplies the orthogonal (Newton-slope) half. **This node is the structural backbone of the partition**; **lem:descent-repeated-factor** alone ($\exists$ one repeated factor) is insufficient.
- *Notation guard* ($d_S \geq 2$, **def:countCellCoeff-ref**). The T_{BB3} residual count **lem:nonsqfree-count** requires residual degree ≥ 2 ; a leaf factor ψ_j with $d_S = 1$ contributes the trivial coefficient $(q^\delta)^{d_S-1} = (q^\delta)^0 = 1$ and is handled *directly* (not via **residualBoxCount**), exactly as flagged in **subsec:notation-v2**, fix (4). Multiplicities a_{ψ_j} are ; the “ $d_S - 1$ ” off-by-one in **countCellCoeff** is in the *exponent*, not in a_{ψ_j} .

7.2 GAP 4 (M3-*i*M7): the per-side L4 lattice face

Definition 7.2 (Per-side L4 lattice face). [**lean**: **LeanUrat.OM.M3.toSideFace**] Fix a monic $f \in [X]$ and a side $S = ((i_0, v_0), (j_0, v_1)) \in \text{npSides}(f)$ (**def:np-vertices**, the labeled side sub-datum **def:newtonpoly-side**) of horizontal length $\ell(S) = j_0 - i_0$ and slope $\lambda_S = -h_S/e_S$ in lowest terms. The *per-side L4 face*

$$\text{toSideFace}(f, S) : \text{L4.LatticePolygon}$$

is the lattice polygon *restricted to the abscissa window* $[i_0, j_0]$ of S — **width** $\ell(S)$, **not the full degree** n — defined by the three **L4.LatticePolygon** fields (**def:to-lattice** target type; **width/ceilHeights/isVertex**, **L4.lean:59--67**):

- **width** $:= \ell(S) = j_0 - i_0$ (the side occupies $[i_0, j_0]$, an interval of width $\ell(S)$; an order- r refinement uses $e = e_S$ as the relevant lattice width via the residual side, but at order 1 the box columns are the $\ell(S)$ integer abscissae of the side);
- **ceilHeights** $t := \lceil \text{npHeight}_f(i_0 + t) \rceil$ for $t \in \{0, \dots, \ell(S) - 1\}$ (**Fin** (**width**)), i.e. **npHeight** of **def:npheight** *restricted to the window* $[i_0, j_0]$, realized as $((\text{npHeight } f \text{ (} i_0 + t \text{)})) \cdot \text{ceil} \cdot \text{toNat}$; because npHeight_f is affine with slope λ_S on the closed side S (**lem:npheight-convex**), these are the per-column closed exponents $\lceil v_0 + \lambda_S t \rceil$ on the side;
- **isVertex** $t := (i_0 + t \in (\text{npVertices } f) \cdot \text{map Prod.fst})$, a **Bool** — but, since S is a *single side* (one segment between consecutive hull vertices), the only window-internal lattice vertex relevant to the box count is the *left* endpoint $t = 0$ (the right endpoint j_0 is the left endpoint of the next side, excluded from this window by the $t < \ell(S)$ range, exactly as **def:to-lattice** excludes abscissa n).

*Flag: NOVEL (the per-side restriction; GMN’s principal Newton polygon of a single side, Def 2.3/§1.1 fact (II)). The target type **L4.LatticePolygon** and the consumers **L4.newtonExponent**, **L4.newtonVertexCount**, **L4.bb1Value**, **L4.columnMeasure**, **L4.cellVolume_eq** are CITED (**ext:L4-cellVolume**; **LeanUrat.L4**, **L4.lean:69--147**). Companion of **def:to-lattice**, which exposes only the full-degree ($\text{width} = n$) polygon; the M7 promise there (“M7 restricts to a side”) is discharged here.*

Lemma 7.3 (Per-side face correctness: **bb1Value** is the per-cell box term). [**lean**: **LeanUrat.OM.M3.toSideFace`bb1Value**] Write $\Pi_S := \text{toSideFace}(f, S)$. Then for every residue size $Q \geq 1$:

1. **L4.newtonExponent** $\Pi_S = \sum_{t=0}^{\ell(S)-1} \lceil \text{npHeight}_f(i_0 + t) \rceil = A_S$, the per-side closed BB1- A exponent (the sum of the column ceilings *over the side window only*);
2. **L4.newtonVertexCount** $\Pi_S = V_S$, the count of side-window vertex columns (in the order-1 single-side case, $V_S = 1$: just the left endpoint i_0);

3. $\text{L4.bb1Value } \Pi_S Q = (1 - Q^{-1})^{V_S} Q^{-A_S}$, which is the per-cell Newton-box term T_{BB1} that `countCellCoeff`'s `L4.bb1Value c.polygon Q` factor evaluates (`def:countCellCoeff-ref`), once `c.polygon :=  $\Pi_S$` is set by `def:cells-of-type`.

Proof sketch. Clauses (1)–(2) are definitional unfolds of `L4.newtonExponent` (`= Finset.sum over Fin width of ceilHeights`, `L4.lean:70--71`) and `L4.newtonVertexCount` (`= Finset.filter (isVertex) | > .card`, `L4.lean:74--75`), substituting the `ceilHeights/isVertex` fields of `def:side-to-l4face` and re-indexing $t \leftrightarrow i_0 + t$ (`Finset.sum_range/Fin.sum_univ_eq_sum_range`, the same mechanical re-indexing as in `lem:bridge-correct` but over the window $[i_0, j_0)$ rather than $[0, n)$). Clause (3) is then *exactly* `L4.bb1Value` by definition (`L4.lean:79--80: bb1Value  $\Pi_S Q = (1 - Q^{-1})^{\text{newtonVertexCount}}$` , `( $Q^{\text{newtonExponent}}$ )-1`), so it is a `rfl/simp` after (1)–(2).

The one content-bearing point is that the `ceilHeights` are the honest per-column exponents of the side: on the closed side S , `npHeightf` is the affine function $x \mapsto v_0 + \lambda_S(x - i_0)$ (`lem:npheight-convex: npHeight` restricted to a single hull side is affine, slope λ_S), so `ceilHeightst =  $\lceil v_0 + \lambda_S t \rceil$` are the BB1 column ceilings $\lceil h_t \rceil$; at the endpoints (lattice vertices) the height is integral ($\lceil h \rceil = h$, the `lem:bridge-correct` integer-vertex fact re-used on the window), collapsing the vertex column's $(1 - Q^{-1}) \cdot Q^{-\lceil h \rceil}$ shell correctly. **Hard point:** the `Fin ( $\ell(S)$ )` vs. `Finset.range` window re-indexing (offset by i_0), mechanical but the source of the off-by-one to watch: the window is half-open $[i_0, j_0)$ of width $\ell(S)$, so the right vertex j_0 is *not* a column of Π_S (it belongs to the next side), mirroring `def:to-lattice`'s exclusion of abscissa n . $\square$

Connection to `lem:l4-coordinate-residue-indep` (M8) and `def:cells-of-type` (M7). `def:cells-of-type` sets each cell's `polygon` field to $\Pi_S = \text{toSideFace}(f, S)$ for the side S carrying that cell's residual stratum, so the per-cell box factor in `countCellCoeff` is the genuine per-side $(1 - Q^{-1})^{V_S} Q^{-A_S}$ rather than the full-degree $(1 - Q^{-1})^V Q^{-A}$ of `def:to-lattice` — the latter is the *product* over sides of the former. Crucially, `lem:l4-coordinate-residue-indep` (M8, the re-proof of `L4.cellVolume_eq`'s `hfactor` hypothesis over $\text{ZMod}(p^N)$, `ext:L4-cellVolume`) factors the count of monic polynomials whose order-1 Newton polygon *on the side window* $[i_0, j_0)$ equals Π_S into the product over the $\ell(S)$ side columns of the per-column `L4.columnMeasure` (`L4.lean:88--90`); without this per-side face there is no per-cell polygon to run that product over, and the cited `L4.cellVolume_eq` (`L4.lean:103--131`, which takes `hfactor` as a hypothesis) cannot be discharged cell-by-cell. Hence Π_S is the precise lattice datum the M8 long pole consumes.

7.3 GAP 2 (M5/M7/M9): the ClusterShape ¡-¿ OMType transport

The five `MontesData` fields are *stated* on the opaque `ClusterShape` index (`structure ClusterShape where tree ×`) deriving `DecidableEq`, `Interface.lean:167--170`; node tuples record (`order, clusterSize, residualDeg`) but all OM content is *proved* on the rich `OMType` (`def:om-type`). GAP 2 supplies the explicit `(de)serializer` that crosses this boundary.

Definition 7.4 (The shape encoder `shapeOf : OMType → ClusterShape`). [`lean: LeanUrat.OM.M5.shapeOf`] The encoder serializes a real OM type into the opaque index:

$$\text{shapeOf}(T) := \langle \text{encodeTree}(T) \rangle \in \text{ClusterShape},$$

where `encodeTree( $T$ ) : List( $\times \times$ )` is the pre-order traversal of the decorated OM tree of T : the root contributes the node tuple $(r, \text{cs}(T), d_r) = (T.\text{order}, \text{clusterSize } T, R_r.\text{natDegree})$ (`def:cluster-size;  $d_r = R_r.\text{natDegree}$` is the A0 tie `hResidualDeg`), followed by the concatenation $(\parallel_{c \in \text{cellsOfType}(T)} \parallel_{ch \in c.\text{children}} \text{encodeTree}(ch.\text{node}))$ over the descent children of every cell (`def:om-cell-child`). This is exactly the data `ClusterShape.tree` was designed to carry

(`Interface.lean:160--169`); it is the count-native counterpart of the dead `PadicMeasure.OMShape` (`ext:OMShape-shadow`), whose `size` field is our root $\text{cs}(T)$ and whose `order/residualDeg` match the root tuple's first/third slots. **NOVEL** (the count-native serializer; replaces the measure route's opaque `OMShape` by an explicit traversal of the real M4/M5 residual data).

Remark 7.5 (Why an explicit serialization, and decidability). `encodeTree` recurses along `ch.node`, whose cluster size is strictly smaller than $\text{cs}(T)$ at every descent (`lem:descend-of-factors`, $\text{cs}(\text{ch.node}) = \mu \geq 2$ but bounded by the parent's degree budget, `thm:treesize-le-n`); so the traversal is a legitimate `termination_by` $\text{cs}(T)$ well-founded recursion (the same measure M6 descends on, `def:descent-measure`), discharged by `InvImage.wf` on `clusterSize`. The codomain `ClusterShape` has `DecidableEq` for free (`derivingDecidableEq` on a `List` of `-tuples`), which is what makes the fiber classifier of `def:shape-class` `DecidablePred` and hence usable in `Finset.sum_fiberwise`.

Definition 7.6 (The level- N shape classifier on decided cosets). [`lean: LeanUrat.OM.M7.shapeClass`] For a fixed degree- n type σ and level N , the *shape classifier* is

$$\text{shapeClass} : \{f \in \text{monicBox}(N, n) : f \text{ decided to type } \sigma\} \longrightarrow \text{ClusterShape}, \quad f \longmapsto \text{shapeOf}(\text{omTree}_N(f))$$

where $\text{omTree}_N(f)$ is the (unique, `lem:om-tree-uniqueness`) OM type produced by the N -truncated Okutsu–Montes algorithm on the digits of f below $\mathfrak{p}^N$ (over the finite residue ring `def:residueRing`, box `def:monicBox`), and `shapeOf` is `def:shape-of-type`. Single-valuedness is exactly the OM uniqueness import `lem:om-tree-uniqueness`; the image lands in $\text{shapesOf}(\sigma)$ because a coset decided to σ has leaf data assembling to σ (GMN Cor 4.19). Decidability of the fibers $\{f : \text{shapeClass}(f) = T\}$ is by `ClusterShape.DecidableEq` (`rem:shape-encode-dec`). This is the map that indexes the strata partition of `lem:m7-partition` via `Finset.sum_fiberwise`, and whose fibers *define* `stratumCount T N` (`def:stratumCount`). **NOVEL** (the classifier the `partition` field's fiber-wise sum runs over; depends on the CITED `lem:om-tree-uniqueness`).

Definition 7.7 (The decoder `decode : ClusterShape  $\rightarrow$  OMType`). [`lean: LeanUrat.OM.M9.decode`] The `MontesData` fields are stated on `ClusterShape` but proved on `OMType`; the decoder is the section that transports proofs back. It rebuilds an OM type from the serialized tree:

$$\text{decode}(S) := \text{rebuild}(S.\text{tree}) : \text{OMType},$$

`rebuild` parsing the pre-order list `S.tree` into the rooted tree, reading each tuple (r, s, d) as a node of order r , cluster size s , residual degree d , and re-attaching the M4/M5 residual/Newton-polygon data canonically by re-running the local OM step at that order (the rich fields r, ϕ_r, N_r, R_r are reconstructed from (r, s, d) together with the M4 residual factorization of the parent at that descent). On the *realizable* image $\text{Im}(\text{shapeOf}) := \{\text{shapeOf}(T) : T \text{ a real OM type of some } f\}$, `decode` is a genuine section: $\text{shapeOf}(\text{decode}(S)) = S$ for $S \in \text{Im}(\text{shapeOf})$, and $\text{decode}(\text{shapeOf}(T)) \cong T$ up to the OM-uniqueness equivalence (`lem:om-tree-uniqueness`: the tree of a fixed f is unique, so the canonical rebuild agrees with the original up to the irrelevant carried ϕ_r , used only through its degree). Off the realizable image `decode` is set to a junk type; this is harmless because every `ClusterShape` fed to a `MontesData` field by M7/M9 lies in $\text{shapesOf}(\sigma) \subseteq \text{Im}(\text{shapeOf})$ by construction (`def:real-montesdata`). **NOVEL** (the count-native section replacing the measure route's implicit `OMShape-to-tree` map; plays for M9 the role `OMShape` played in `Interface.OMBridge`, cf. `rem:shape-shadow`).

Lemma 7.8 (Transport: the five fields proved on `OMType` descend to `ClusterShape`). [`lean: LeanUrat.OM.M9.shape'transport`] Defining the `MontesData` fields on `ClusterShape` by pulling back along `decode` — $\text{treeSize}_D(S) := \text{cs}(\text{decode } S)$ and $\text{cells}_D(S) := \text{cellsOfType}(\text{decode } S)$ (`def:cells-of-type`) — the three transport identities hold:

1. **size transport:** $\text{treeSize}_D(\text{shapeOf } T) = \text{cs}(T)$ for every real T (`def:cluster-size`; the root tuple's `clusterSize` slot is preserved by `encodeTree/rebuild`);
2. **cells transport:** $\text{cells}_D(\text{shapeOf } T) = \text{cellsOfType}(T)$ (because `decode` $\circ$ `shapeOf` fixes a type up to OM-equivalence and `cellsOfType` depends on T only through N_r and the residual factorization, both uniqueness-pinned by `lem:om-tree-uniqueness`);
3. **children commute:** for every cell $c \in \text{cellsOfType}(T)$ and child $ch \in c.\text{children}$, the `ClusterShape` child stored in $c.\text{children}$ (recall `CountCell.children : List ClusterShape`, `MontesAxiom.lean:108`) equals $\text{shapeOf}(ch.\text{node})$, so $\text{treeSize}_D(ch) = \text{cs}(ch.\text{node}) = \mu < \text{cs}(T) = \text{treeSize}_D(\text{shapeOf } T)$ (`lem:descend-of-factors`, `lem:child-size-lt`). This is the identity that turns the `OMType`-level strict descent into the `MontesData.cells_descend` statement (`lem:cells-descend`).

NOVEL (the load-bearing transport; without it `cells_descend`, `finiteTermination`, and `partition` — all quantified over `ClusterShape` — are not connected to their `OMType` proofs).

Proof sketch. All three are unfoldings of `encodeTree/rebuild` against the field definitions, modulo the OM-uniqueness pin. (1) `encodeTree(T)` places $(T.\text{order}, \text{cs}(T), R_r.\text{natDegree})$ as its head; `rebuild` reads the head's second slot back as the root cluster size, so $\text{cs}(\text{decode}(\text{shapeOf } T)) = \text{cs}(T)$ by `List.head` computation and the A0 tie `clusterSize = cs (def:cluster-size)`; cast through the `rfl` definition of treeSize_D . (2) `cellsOfType (def:cells-of-type)` is a function of $(N_r, \text{the residual factorization } R_S = \prod_{\psi} \psi^{a_{\psi}})$; both are determined up to the GMN-unique dissection (`lem:om-tree-uniqueness`), and ϕ_r enters only via its degree m_r , which is recorded; hence $\text{cellsOfType}(\text{decode}(\text{shapeOf } T)) = \text{cellsOfType}(T)$ by `congrArg` on the pinned data. (3) By construction `cellsOfType` stores each child as $\text{shapeOf}(ch.\text{node})$ (`def:cells-of-type` sets `children := the singleton [shapeOf( $T_{\psi}$ )] at a descent`), so the child slot is a `ClusterShape` and treeSize_D of it is $\text{cs}(ch.\text{node})$ by (1); the strict inequality is `lem:descend-of-factors` ($\mu \geq 2$) combined with the M6 degree-budget bound $\mu \leq \deg \psi \cdot a_{\psi} \leq \deg R_S \leq \text{cs}(T)$ off the self-loop (`lem:child-size-lt`). The recursion terminates because every appeal to (1)/(3) at a child is at strictly smaller `clusterSize` (`InvImage.wf`, `rem:shape-encode-dec`). No new mathematics beyond the serializer's round-trip and the cited OM uniqueness. $\square$

7.4 GAP 3 (M5): the CITED GMN dissection + OM-tree uniqueness

Lemma 7.9 (OM dissection is a disjoint exhaustive partition; the OM tree is unique). [`lean: LeanUrat.OM.M5.om`tree`unique`] **Status:** **CITED (import, not a novel proof)**. Sources: GMN (`arXiv:0807.2620v2`; TAMS 364 (2012)) **Theorem 3.1** (Theorem of the polygon in order r) and **Theorem 3.7** (Theorem of the residual polynomial in order r), with the leaf/descent dichotomy **Corollary 3.8** (`lem:om-leaf-descent`), the descent trigger **Lemma 3.11(3)**, and the full factorization **Definition 3.10** / **eq. (37)**; cf. `notes/GMN_citations.md` §2.2–2.3. The two order- r dissections refine the p -adic factorization of a fixed monic separable f into pairwise-disjoint, exhaustive strata, and the resulting Okutsu–Montes tree of f is unique:

1. **(Slope dissection, Thm 3.1).** If $\omega_r(f) > 0$ and the order- r principal Newton polygon $N_r^-(f) = S_1 + \cdots + S_g$ has g sides of *pairwise different* slopes $\lambda_{r,1}, \dots, \lambda_{r,g}$ (`def:newtonpoly-side`), then $f_t = F_1 \cdots F_g$ with $N_r(F_i)$ one-sided of slope $\lambda_{r,i}$ and $R_{\lambda_{r,i}}(F_i) \sim R_{\lambda_{r,i}}(f)$. Distinct slopes split off *coprime* factors.
2. **(Residual dissection, Thm 3.7).** For a side S of finite slope λ_r with residual factorization $R_{\lambda_r}(f) \sim \psi_{r,1}^{a_1} \cdots \psi_{r,t}^{a_t}$ into *pairwise-distinct* monic irreducibles in $F_r[Y]$ (`lem:residual-factorization`),

the factor $f_{t,\lambda_r} = G_1 \cdots G_t$ with $N_r(G_i)$ one-sided of slope λ_r and $R_{\lambda_r}(G_i) \sim \psi_{r,i}^{a_i}$. Distinct residual factors split off *coprime* factors.

3. (**Dichotomy + trigger, Cor 3.8 / Lemma 3.11(3)**). A multiplicity-1 residual factor ($a_i = 1$) yields an *irreducible* p -adic leaf with explicit $(e, f) = (e_1 \cdots e_r, f_0 \cdots f_r)$; a multiplicity- ≥ 2 factor descends to order $r + 1$. The descent at order r fires *iff* some order- r residual is non-separable (= non-squarefree).
4. (**Disjoint exhaustive partition + uniqueness, Def 3.10/eq. (37), Cor 4.19**). Iterating the two dissections through all orders, Hensel's lemma gives $f = f_{r,\infty} \cdot \prod_{t \in T_r(f)} f_t$, a partition of the irreducible p -adic factors of f into pairwise-disjoint strata indexed by the OM types t ; since at each order the dissection is by pairwise-*different* slopes and pairwise-*distinct* (coprime) residual factors, the strata are disjoint and exhaustive. The associated decorated OM tree of f is therefore **unique**: two completed OM trees of the same f agree as trees (same nodes, slopes, residual factors, multiplicities), up to the irrelevant carried key-polynomial data ϕ_r (used only through its degree).

This is an **IMPORT** of existence/uniqueness; we do *not* reprove GMN's theorems. In Lean it is carried as the named hypothesis bundle `om_tree.unique` (the p -independent structural content of Thm 3.1 + Thm 3.7 + Cor 3.8 + Lemma 3.11(3) + eq. (37)), distinct from the surviving analytic axiom `tame_functionalEquation` (`def:surviving-axiom`); it is a combinatorial factorization fact, p -independent, treating the wild/inseparable case as generic (`notes/GMN_citations.md` §0, facts (I)–(III)).

Proof sketch. Cited, not reproved. The mathematical content is GMN Theorems 3.1 and 3.7 (the order- r analogues of Ore's Theorems of the polygon and of the residual polynomial), whose pairwise-distinctness hypotheses (distinct slopes; distinct monic-irreducible residual factors) make the split factors *coprime*, hence the strata *disjoint*; exhaustiveness is the completeness statement of Def 3.10/eq. (37) (every irreducible p -adic factor of f lies in exactly one f_t , possibly after finitely many descents, terminating by Cor 4.19). Uniqueness of the tree is then a formal consequence: disjoint + exhaustive forces the node set, slopes, and residual factors to be invariants of f , and the coprime split makes the assignment of each p -adic factor to a stratum single-valued. The one place *we* contribute is feeding the residual-side hypothesis from `lem:residual-factorization` (the complete coprime factorization $R_S = c \prod_{\psi} \psi^{a_{\psi}}$ over the finite F_r), which is exactly the “pairwise-distinct monic irreducibles” input Thm 3.7 requires. No `mathlib` lemma is invoked here; this node is the GMN boundary itself. $\square$

Remark 7.10 (How `lem:m7-partition` consumes the uniqueness import). `lem:m7-partition` (discharging `MontesData.partition`, $M.\text{decidedCount } \sigma N = \sum_{T \in \text{shapesOf}(\sigma)} \text{stratumCount } T N$) consumes `lem:om-tree-uniqueness` at two points, both routed through the classifier `shapeClass` (`def:shape-class`) and the transport `lem:shape-transport`:

1. **Well-definedness of the fiberwise sum.** `Finset.sum_fiberwise` (equivalently `Finset.card_eq_sum_card`) requires a *single-valued* classifier `shapeClass`. Single-valuedness is precisely clause (4) of `lem:om-tree-uniqueness`: the N -truncated OM tree `omTreeN(f)` of a fixed decided coset f is unique, so `shapeOf(omTreeN(f))` is a well-defined element of `shapesOf(σ)` (image lands in the right `Finset` by Cor 4.19).
2. **Disjointness/exhaustiveness of the strata.** The fibers $\{f : \text{shapeClass}(f) = T\}$ partition the decided set because the dissections (1)–(2) of `lem:om-tree-uniqueness` are by

pairwise-distinct slopes and coprime residual factors, the latter supplied at each level by `lem:residual-factorization`. The fiber cardinalities are then `stratumCount T N` by `def:stratumCount`, and `lem:shape-transport(2)–(3)` identifies the `ClusterShape`-indexed `cellsD(T)` used downstream with the `OMType`-built `cellsOfType`.

Thus the only nontrivial inputs to `partition` are the CITED OM uniqueness/dissection (`lem:om-tree-uniqueness`) and the complete residual factorization (`lem:residual-factorization`); the remainder is finite `Nat.card` combinatorics over `monicBox(N, n)` cast to $\mathbb{Q}$ (`Nat.cast_sum`). **NOVEL bookkeeping.**

7.5 GAP 5 (M8): the finite certification level $N_0(c)$

Lemma 7.11 (The finite certification level $N_0(c)$). [`lean: LeanUrat.OM.M8.certLevel`] Fix the monic $f = X^n + \sum_{i < n} a_i X^i \in O_K[X]$ ($a_i \in O_K =_{[p]}$ in the core $K =_p$ case, so $q = p$) and a cell $c = (S, \psi) \in \text{cellsOfType}(T)$ (`def:cells-of-type`). There is a *finite* level $N_0(c)$: — explicitly definable, well-defined, depending only on f and c (not on the truncation level N) — such that for every $N \geq N_0(c)$ the following data of f are *determined by, and stable under, the reduction* $f \bmod \mathfrak{p}^N$:

1. the lower Newton-polygon vertices and the abscissae/slope of the side S (`def:np-vertices`): each finite valuation $v_i = v(a_i)$ that names a vertex of N_r satisfies $v_i < N_0(c)$, so v_i is read off correctly from $a_i \bmod \mathfrak{p}^{N_0(c)}$ (a coefficient with $v(a_i) \geq N$ is indistinguishable from 0 $\bmod \mathfrak{p}^N$, but the *vertex* valuations are all $< N_0(c)$ by construction);
2. the residual polynomial $R_S \in F_r[Y]$ along S (`def:residual-poly`), in particular its leading and trailing coefficients and hence the ψ -multiplicity a_ψ in $R_S \sim \prod_\psi \psi^{a_\psi}$ (`lem:residual-factorization`): finite-field data extracted from the bottom digit of the side coefficients, fixed once the relevant a_i are known $\bmod \mathfrak{p}^{N_0(c)}$.

Concretely $N_0(c) := 1 + \max(\{v_i + 1 : i \in \text{vertices}(N_r)\} \cup \{\text{side-coefficient precision for } R_S\})$, a finite max over the *finite* support set `valSupport(f)` (`def:val-support`). For $N \geq N_0(c)$ the per-step fiber ratio of `lem:constant-fiber` is *exactly* constant (no transient correction term), so $N_0(c)$ is precisely the lower quantifier bound under which `lem:constant-fiber` and `thm:levelN-bijection` are stated. **NOVEL** (makes the “for $N \geq N_0(c)$ ” precondition of the M8 long pole a defined, finite, well-defined object).

Proof sketch. Finiteness / well-definedness. Each Newton-polygon vertex is a coefficient abscissa i with $a_i \neq 0$ and finite $v_i < \infty$; the set of such abscissae is a subset of the finite `valSupport(f) := \{i < n : a_i \neq 0\}` (`def:val-support`, a `Finset(Fin n)` with $\# \leq n$). A finite set of naturals has a `max(Finset.max' / Finset.sup)`, so $N_0(c)$ is a finite natural number; the max over empty support degenerates harmlessly to the residual-precision term.

Stabilization (the load-bearing half). Write each $a_i \in O_K$ as a p -adic integer with level- N natural approximant `appr ai N` (`PadicInt.appr, RingHoms.lean:354`). The spec `PadicInt.appr_spec N : ai - appr ai N ∈ Ideal.span{pN}` (`RingHoms.lean:403`) together with `appr_lt` (`appr ai N < pN, :363`) and `appr_mono` (`:381`) say: the low-order digits of a_i are fixed once N exceeds them, i.e. for $N \geq N'$ the residue $a_i \bmod \mathfrak{p}^N$ refines (does not alter) $a_i \bmod \mathfrak{p}^{N'}$ via `PadicInt.zmod_cast_comp_toZModPow` (`:476`, the projective-system compatibility $\pi_{N'}^N \circ \text{toZModPow}_N = \text{toZModPow}_{N'}$). A valuation v_i is the position of the lowest nonzero digit, present in $a_i \bmod \mathfrak{p}^N$ once $N > v_i$ (it lies in $\ker \text{toZModPow}_N$ only if $N \leq v_i$, by `PadicInt.ker_toZModPow, :457`, `kernel = span{pN}`). Taking $N_0(c)$ one above the largest *vertex* valuation and above the side’s residual precision makes (1) and

(2) read correctly from $f \bmod \mathfrak{p}^{N_0(c)}$, and `appr_mono` makes them *stable* for all $N \geq N_0(c)$. Coefficients with $v_i \geq N_0(c)$ (the high-valuation tail) sit strictly above the side and never name a vertex of N_r , so they are irrelevant to c — this is why a *finite* $N_0(c)$ suffices despite O_K being infinite. Mathlib tools: `appr_spec`, `appr_lt`, `appr_mono`, `ker_toZModPow`, `zmod_cast_comp_toZModPow`, and `Finset.le_sup / Finset.max'_mem` for the max over `valSupport(f)`. $\square$

7.6 GAP 6 (M8): the self-loop ratio resums to `countPivot`

Lemma 7.12 (The self-loop ratio resums to `countPivot`). [lean: `LeanUrat.OM.M8.selfLoopRatioEq`] Among the cells of a real shape T there is a *unique self-loop cell* — the cell whose stratum reproduces the full parent cluster (the multiplicity- ≥ 2 residual factor ψ with $R_S = \psi^{a_\psi}$, $\deg \psi = 1$, $a_\psi = \text{cs}(T)$; the non-progressing cell isolated in `lem:child-size-lt` that does *not* emit a strictly smaller child). Write $s := \text{treeSize } T$ for the parent cluster size and $w := \text{selfLoopExponent}(s) = s(s+1)/2 - 1$ (`L5fix.selfLoopExponent`, NOT the Newton side ramification e_S ; `subsec:notation-v2`, fix (3)). Then:

1. the per-step constant-fiber ratio (`lem:constant-fiber`) of the self-loop cell is exactly

$$r_{\text{loop}} = q^{-w} = ((q :)^{\text{selfLoopExponent}(s)})^{-1}, \quad s = \text{treeSize } T;$$

2. consequently the geometric tail $\sum_{N \geq 0} r_{\text{loop}}^N = 1/(1 - q^{-w})$ converges for $q \geq 2$ to the reciprocal of the pivot,

$$\sum_{N \geq 0} (q^{-w})^N = \frac{1}{1 - q^{-\text{selfLoopExponent}(s)}} = (\text{countPivot } (\text{treeSize } T) q)^{-1} \quad (s \geq 2),$$

and the self-loop contributes the trivial factor 1 when $s < 2$ (the leaf branch of `countPivot`).

The argument fed to `L5fix.selfLoopExponent` is `treeSize T` (the cluster size s , the “ e ” of `selfLoopExponent`), matching the branch $2 \leq s$ of `countPivot` (`MontesAxiom.lean:126--127`) verbatim — *not* the Newton side ramification e_S . **NOVEL** (the explicit node tying $r(c)$ of `lem:constant-fiber` to `countPivot`).

Proof sketch. (1) *Identifying the exponent (watch the off-by-one in $w(s)$).* On the self-loop cell the cluster neither splits nor shrinks: refining $N \rightarrow N + 1$, the within-cluster lifts that stay together contribute the within-block `box×Vandermonde` rescale $q^{-(s+\binom{s}{2})} = q^{-\text{blockExponent}(s)}$ of `NestedCollapse.blockExponent` ($= s + s.\text{choose } 2 = s(s+1)/2$), but *one* of those slots is the already-fixed certified digit of `lem:certification-level` (the cell’s own residual unit, certified at $N_0(c)$ and not re-counted), so the genuine *free* per-step ratio is $q^{-(\text{blockExponent}(s)-1)}$. This is exactly the off-by-one: by `RescaleC1.selfLoopExponent.eq_blockExponent_sub_one` (`MontesAxiom.lean:683`, PROVED) $\text{selfLoopExponent}(s) = \text{blockExponent}(s) - 1$, so $r_{\text{loop}} = q^{-\text{selfLoopExponent}(s)} = q^{-w}$ with $w = s(s+1)/2 - 1$ (sanity: $s = 2 \Rightarrow w = 2$, $s = 3 \Rightarrow w = 5$). `lem:constant-fiber` supplies that this ratio is N -independent for $N \geq N_0(c)$. *Crucially the argument is $s = \text{treeSize } T$, the cluster size, not e_S :* the geometric collapse counts the ways the s cluster members redistribute, a function of s alone.

(2) *Geometric resummation to the pivot.* Set $a := 1$, $r := q^{-w} = ((q :)^{\text{selfLoopExponent } s})^{-1}$. For $q \geq 2$ and $s \geq 2$, $\text{selfLoopExponent } s \geq 1$, so $0 < r < 1$: positivity and $1 - r > 0$ are exactly `L5fix.selfLoop.geometric s q` ($2 \leq s$) ($2 \leq q$) (`L5fix.lean:43`). The finite truncations $V_N = \sum_{j=0}^N r^j$ (`NestedCollapse.geomTrunc.eq_sum`, :373) converge to $a/(1 - r) = 1/(1 - q^{-w})$

by `NestedCollapse.geometricLimit_of_selfLoop a r (0 < r) (r < 1) (:386)`. By definition $\text{countPivot } s \, q = 1 - ((q :)^{\text{selfLoopExponent } s})^{-1} = 1 - r$ for $2 \leq s$ (`MontesAxiom.countPivot, :126--127`), so the limit is $1/(1-r) = (\text{countPivot } s \, q)^{-1}$; pole-free at $q > 1$ by `MontesAxiom.countPivot_ne (:187, via selfLoop_geometric)`. For $s < 2$ there is no self-loop cell and the factor is the constant $1 = \text{countPivot } s \, q$. This is the tie consumed by `thm:stratum-limit(2)`: the “ $1/\text{countPivot}(\text{treeSize } T) \, q$ ” denominator IS the resummed self-loop geometric series. $\square$

7.7 Canonicalization of labels (v1 -i v2)

Applying the renames below makes every body edge resolve against a single canonical . The left column is the drifted label(s) appearing in the v1 body; the right column is the canonical slug. Four small canonicalization-glue nodes are added at the same time (rows marked **[new node]**): they are pure cite/alias nodes the body already references.

The four **[new node]** stubs are stated here for completeness (each a one-line CITED cite or a split-out of an existing definition):

Definition 7.13 (External: the L4 cell-volume identity (cite)). [`lean: LeanUrat.L4.cellVolume`eq`] **Status: CITED (external, proved)**. Source: `L4.lean:103--131`. The lattice box count  $\text{cellVol pg } Q = \text{bb1Value pg } Q$  given the factorization hypothesis  $\text{hfactor} : \text{cellVol pg } Q = \prod_{i:\text{Fin width}} \text{columnMeasure pg } Q \, i$ . The single external-cite label for both M3 (`def:to-lattice/def:side-to-l4face`) and M8 (`lem:l4-coordinate-` which *discharges* the `hfactor` hypothesis over  $\text{ZMod}(p^N)$ ). Replaces the v1 drifted labels `lem:l4-cellvolume` (layer 3) and `L4-cellVolume_eq` (layer 1).

Definition 7.14 (External: the dead-measure `OMShape` shadow (cite)). [`lean: LeanUrat.PadicMeasure.OMShape`] **Status: CITED (external, DEAD measure route, mirrored for field layout only)**. Source: `PadicMeasure` (referenced, bypassed). `OMShape` carries `(size, order,  $r$ ,  $0$ , residueDeg,  $N_r$ , residual, resid)` with the two A0 ties; the count-native `OMType` (`def:om-type`) refines it with honest M4 residual data (`rem:shape-shadow`). The measure route (`clusterMeasure`, `AX_cellRecursion`, `omCells`, `descend.size_lt`) is bypassed entirely; this node exists only so the v1 dangling cite `ax:om-shape-shadow` resolves.

Definition 7.15 (The labeled Newton side sub-datum). [`lean: LeanUrat.OM.M3.npSide`] **Status: NOVEL (split-out of `def:np-vertices`)**. A *side* of the lower Newton polygon is the segment $S = ((i_0, v_0), (j_0, v_1))$ between two consecutive hull vertices of `def:np-vertices`, with horizontal length $\ell(S) = j_0 - i_0$ and slope $\lambda_S = -h_S/e_S = (v_1 - v_0)/(j_0 - i_0)$ in lowest terms (e_S the Newton-side ramification denominator). `npSides(f) : List` is the list of these. This is the “Side” sub-datum the v1 body referenced as `def:newtonpoly-side`; it is split out of `def:np-vertices` so that `def:side-to-l4face`, `def:cells-of-type`, `lem:om-tree-uniqueness` can it by name.

Lemma 7.16 (The leaf/descend dichotomy of an OM cell). [`lean: LeanUrat.OM.M5.leaf`or`descend`] **Status: NOVEL (named dichotomy, fusing `def:om-cell` and `lem:descent-repeated-factor`)**. For each cell $c = (S, \psi)$ of an OM type (`def:om-cell`), exactly one of: (*leaf*) $a_\psi = 1$, `c.children` = `[]`, and c is an irreducible p -adic leaf (GMN Cor 3.8); or (*descend*) $a_\psi \geq 2$, `c.children` = `[shapeOf( $T_\psi$ )]` with T_ψ the order- $(r+1)$ child of cluster size a_ψ (the repeated factor of `lem:descent-repeated-factor` extracted from the full factorization `lem:residual-factorization`). This is the single named node the v1 body referenced as `lem:om-leaf-descent`; it underlies `def:cells-of-type`’s children case split and `lem:cells-descend`.

Front-matter forward stubs. The five v1 forward-target labels (`def:m6-termination-target`, `def:m7-cells-target`, `def:m8-partition-target`, `def:m8-boxhaar-target`, `def:m8-boxsum-target`)

and `def:m1-box-card` are resolved by option (b) of the v1 reconciler: the front-matter lists now point at the real downstream slugs — respectively `thm:treesize-le-n/thm:cells-descend-discharge/thm:finite` (M6), `def:cells-of-type+lem:m7-partition` (M7), `lem:m7-partition`, `thm:stratum-limit` (parts 1 and 2), and `lem:card-monicBox` (M1). No alias `s` are introduced.

7.8 Notation fixes (v2)

v1 drifted label(s)	canonical v2 slug	note
<code>lem:card-monicbox</code>	<code>lem:card-monicBox</code>	case collision (M1)
<code>lem:m0-residual-count</code>	<code>lem:nonsqfree-count</code>	M0 residual oracle
<code>def:quotient-box</code>	<code>{def:residueRing, def:monicBox}</code>	split pair (M1)
<code>def:newton-polygon</code>	<code>def:np-vertices</code>	M3 polygon
<code>def:lattice-polygon-bridge</code>	<code>def:to-lattice</code>	M3 L4 bridge
<code>lem:residual-sep-iff-sqfree</code>	<code>thm:residual-sep-sqfree</code>	M4 separability
<code>def:omtype</code>	<code>def:om-type</code>	M5 type
<code>def:om-cells-data</code>	<code>def:om-cell</code>	M5 cell
<code>lem:treesize-le-n</code>	<code>thm:treesize-le-n</code>	M6 termination
<code>lem:card-monicBox (case)</code>	<code>lem:card-monicBox</code>	one canonical M1 card
<code>lem:m8-stratumCount-decided</code>	<code>def:stratumCount</code>	M7 partition fix
<code>lem:l4-cellvolume / L4-cellVolume.eq</code>	<code>ext:L4-cellVolume</code>	[new node] one external cite
<code>def:newtonpoly-side</code>	<code>def:newtonpoly-side</code>	[new node] split from <code>def:np-</code>
<code>lem:om-leaf-descent</code>	<code>lem:om-leaf-descent</code>	[new node] named leaf/descend
<code>ax:om-shape-shadow</code>	<code>ext:OMShape-shadow</code>	[new node] CITED dead meas
<code>def:m6-termination-target</code>	<code>thm:treesize-le-n</code> + <code>thm:cells-descend-discharge</code> + <code>thm:finite-termination-discharge</code>	front-matter forward stub
<code>def:m7-cells-target</code>	<code>def:cells-of-type + lem:m7-partition</code>	front-matter forward stub
<code>def:m8-partition-target</code>	<code>lem:m7-partition</code>	front-matter forward stub
<code>def:m8-boxhaar-target</code>	<code>thm:stratum-limit (part 1)</code>	front-matter forward stub
<code>def:m8-boxsum-target</code>	<code>thm:stratum-limit (part 2)</code>	front-matter forward stub
<code>def:m1-box-card</code>	<code>lem:card-monicBox</code>	front-matter forward stub

1. $q = p$ **core (unramified)**, so $(p^N)^n = q^{nN}$. (*State prominently.*) The shared notation has $q = \#(\text{residue field})$ (a power of p) and box count $(p^N)^n$, while `CountingModel.box_partition` (`CountingModel.lean:118`) uses the box total $(q :)^{nN} = q^{nN}$. These agree *only* in the absolutely-unramified core $K =_p, O_K =_{[p]}$, where $q = p$; the formalization fixes this core (`def:residueRing`), so $(p^N)^n = q^{nN}$ and the M1 cardinality `lem:card-monicBox` matches `box_partition`. Outside the core ($q \neq p$) the two box counts disagree and the appr/digit machinery of `lem:certification-level` would need the O_K projective system. Canonical notation-table row: “ $q = p$ (unramified core; box $(p^N)^n = q^{nN}$).”
2. **D.coeff lives in Decomposition, not MontesAxiom.** The `MontesData` structure has no `coeff` field; it has `D.C` (`MontesAxiom.lean:~520`), and the alias `D.coeff` is defined in `Decomposition.lean:70` with `coeff_eq_C : D.coeff T q' = D.C T q'` by `rfl` (`Decomposition.lean:74`). `Goal.goal_theorem_montes` (`Goal.lean:161`) and `countingDensity_eq_sum_coeff` (`Decomposition.lean:105`) both use `D.coeff`, so the blueprint’s $g_\sigma(q') = \sum_T D.coeff T q'$ is correct *provided* one notes `coeff` is the `Decomposition` alias of `C`.
3. e_S (**Newton-side ramification**) vs. $w := \text{selfLoopExponent}(s)$ (**cluster size**). These are different “ e ”s and must not be conflated. e_S is the slope denominator of a Newton side (`def:newtonpoly-side/def:np-vertices`), $\lambda_S = -h_S/e_S$. The argument of `L5fix.selfLoopExponent(e) :=`

$e(e+1)/2-1$ (L5fix.lean:38) is instead the *cluster size* $s = \text{treeSize } T$, which `countPivot` s feeds in its $2 \leq s$ branch (MontesAxiom.lean:126--127). Rename the latter $w := \text{selfLoopExponent}(s)$ throughout and reserve e_S strictly for the Newton denominator (`lem:selfloop-ratio` states the argument is $\text{treeSize } T$, NOT e_S). Watch the off-by-one in $w(s) = s(s+1)/2-1$ (pinned to the PROVED `RescaleC1.selfLoopExponent_eq.blockExponent.sub_one`, MontesAxiom.lean:683; $s = 2 \Rightarrow w = 2$, $s = 3 \Rightarrow w = 5$).

4. **residualBoxCount** needs $d_S \geq 2$ (descent cells); $d_S = 1$ leaf cells handled directly. **residualBoxCount** (MontesAxiom.lean:235) requires `hd : 2 ≤ dS`: the “non-squarefree count = Q^{m-1} ” reading (`lem:nonsqfree-count`) holds only for $m = d_S \geq 2$. `countCellCoeff` uses $(q^\delta)^{d_S-1}$ uniformly; for a leaf cell ($d_S = 1$, a multiplicity-1 irreducible factor) this gives $(q^\delta)^0 = 1$, the correct count of monic degree-1 configurations, but it is *not* the **residualBoxCount** lemma. So leaf cells ($d_S = 1$) are handled *directly* ($= 1$), *not* via **residualBoxCount**; only descent cells ($d_S \geq 2$) invoke the M0 oracle (`lem:residual-factorization`’s “descent factors $a_{\psi_j} \geq 2$ ” vs. “leaf factors $a_{\psi_j} = 1$ ” carries this through).
5. `CountCell.children : List ClusterShape` (so `cellsOfType` emits `ClusterShape` via `shapeOf`, **not** `OMType`). `CountCell.children` is `List ClusterShape` (MontesAxiom.lean:108), and `MontesData.cells_descend` (MontesAxiom.lean:402) quantifies `treeSize ch` with `ch : ClusterShape`. The M5/M7 child datum `OMCell.Child (def:om-cell-child)` carries `ch.node : OMType`; therefore `def:cells-of-type` must emit each child as `shapeOf(ch.node)` (`def:shape-of-type`), i.e. `children : List ClusterShape` via `decode/shapeOf`, *not* `List OMType`. The descent inequality is then on the `treeSize` of that `ClusterShape`, intertwined with the `OMType` proof by `lem:shape-transport(3)`.

A Status (v2 closure verdict)

S.1 DAG closure verdict

DAG-closed and ready to start Lean-ing. With the six glue nodes written in full (GAP 1 lem:residual-fac

S.2 Residual scope boundaries (not DAG gaps)

No structural (DAG-closure) gaps remain: with the six glue nodes plus the four canonicalization-glue st

S.3 Lean-ing order (from v1 reconciliation R.6)

Lean-ing order (bottom-up, with the blueprint gaps that MUST be added as nodes first marked [ADD]):

Phase A (Layer 0, lowest risk, start here):

1. M0 ResidueCount: lem:nonsqfree-count (= re-export residualBoxCount, requires $2 \leq dS$), lem:nonsep-iff
2. M1 QuotientBox: def:residueRing, lem:ker-surj, lem:card-residueRing, def:monicBox, then lem:card-de
3. M2 PadicValuation: def:coeff-val, lem:val-facts, def:val-support, lem:val-support-facts.

Phase B (Layer 1, first genuine geometry):

4. M3 NewtonPolygon: make def:np-vertices (integer cross-product hull scan) PRIMITIVE; derive lem:nphe
5. M4 ResidualPolynomial: def:residual-coeff, def:residual-poly, lem:residual-endpoints, lem:residual-c

Phase C (Layer 2, recursion + termination):

6. M5 OMType: ext:OMShape-shadow (cite node), def:residue-tower, lem:card-Fr, def:om-type, def:cluster
7. M6 Termination: def:descent-measure, lem:child-size-lt, thm:treesize-le-n, then discharge thm:cells

Phase D (Layer 3, the payoff; M8 is the long pole | schedule the most time here):

8. M7 CellPartition: def:cells-of-type, lem:cells-descend, lem:m7-partition (needs GAP 1 + GAP 3).
9. M8 LevelNCount: def:stratumCount, [ADD] lem:certification-level NO(c) (GAP 5), lem:l4-coordinate-re
10. M9 RealInstance: def:real-countingmodel, def:real-montesdata, lem:sep-tail, lem:disc-tail (DEFERRE

Recommendation: do Phase A+B fully (low risk, validates the L3/L4 reuse and the field-vs-ring fallback)